

Minho University
Engineering School

Dong Xuyong

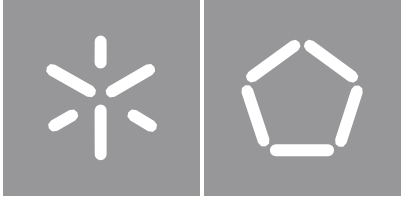
Hierarchical structure optimization for anomaly detection in grouped time series data

Hierarchical structure optimization
for anomaly detection in grouped time series data

Dong Xuyong

UMinho | 2026

October 2026



Minho University
Engineering School

Dong Xuyong

Hierarchical structure optimization for anomaly detection in grouped time series data

Master's dissertation in Information Systems Engineering and Management

Work carried out under the supervision of
Professor Doctor Paulo Alexandre Ribeiro Cortez
Professor Doctor Emanuel Demétrio Mendes Gouveia

October 2026

Copyright and terms of use of the work by third parties

This is an academic work that can be used by third parties as long as internationally accepted rules and good practices are respected with regard to copyright and related rights. The work may be used under the terms of the licence below. If a use is not covered by that licence, the user should contact the author through the RepositóriUM of the University of Minho.

Licence granted to users of this work

Attribution-NonCommercial

CC BY-NC

`https://creativecommons.org/licenses/by-nc/4.0/`

Acknowledgements

Looking back on the work that led to this dissertation, I am deeply grateful to the people whose guidance, support and encouragement made it possible.

My deepest thanks go to my supervisor, Professor Doctor Paulo Alexandre Ribeiro Cortez, who has been a constant source of motivation throughout this research. His knowledge, rigour and dedication to academic excellence guided my work at every stage, and I learned a great deal from his teaching and advice during the master's programme and the whole period of this study.

I am also grateful to my co-supervisor, Professor Doctor Emanuel Demétrio Mendes Gouveia, for his insights and encouragement. His perspective and feedback were essential in shaping and broadening the scope of this work, and the collaborative atmosphere he fostered contributed greatly to my academic growth.

I also thank the telecommunications operator that made the network data available, without which this study would not have been possible, and the developers of the open-source tools on which this work is built, in particular PonyGE2 and STUMPY.

Finally, I thank my family and friends for their unconditional love, patience and encouragement throughout this journey.

Statement of integrity

I hereby declare having conducted this academic work with integrity. I confirm that I have not used plagiarism or any form of undue use of information or falsification of results along the process leading to its elaboration. I further declare that I have fully acknowledged the Code of Ethical Conduct of the University of Minho.

Abstract

This dissertation focuses on anomaly detection in grouped telecommunications time-series data from a real network. Modern telecommunications networks generate large volumes of traffic that must be monitored to detect anomalous behaviour that can affect service reliability, capacity provisioning, and operational decision-making. These datasets are structured along several categorical dimensions, which creates many possible aggregation levels and makes it difficult to decide which time series to analyse. This work proposes a framework based on hierarchical aggregation structures, with the detector held fixed. A single global view or fully disaggregated monitoring is often inadequate: a global view can hide local anomalies, while full disaggregation is costly and can leave many series too sparse for reliable detection. Instead, this work explores partial hierarchical structures that may reveal abnormal traffic patterns at a relevant level of detail, aiming to detect anomalies in complex grouped datasets without sacrificing interpretability or incurring prohibitive analytical cost.

A formal grammar automatically generates hierarchical structures. Each candidate is scored with the Matrix Profile, and the area under the ROC curve (AUC) of those scores is the fitness of the search. In addition to predefined hierarchies, the method can discover useful aggregation structures automatically. To this end, Grammatical Evolution (*Grammatical Evolution* (GE)) (O'Neill and Conor Ryan, 2001; O'Neill and C. Ryan, 2003) is used. The main contribution is a reproducible, interpretable framework for searching aggregation structures with a fixed detector for telecommunications time series. The evolved hierarchies reached a test AUC of 0.678 ± 0.066, roughly 0.10 to 0.14 above the fully aggregated series and the Call, Event, and Service Type splits ($p = 0.002$).
Keywords: anomaly detection, telecommunications, hierarchical time series, grouped time series, Grammatical Evolution, Matrix Profile.

Resumo

Esta dissertação foca-se na deteção de anomalias em dados de séries temporais agrupadas de telecomunicações provenientes de uma rede real. As redes de telecomunicações modernas geram grandes volumes de tráfego que devem ser monitorizados para detetar comportamento anómalo que pode afetar a fiabilidade do serviço, o aprovisionamento de capacidade e a tomada de decisão operacional. Estes conjuntos de dados estão estruturados ao longo de várias dimensões categóricas, o que cria muitos níveis de agregação possíveis e dificulta a decisão sobre quais séries temporais analisar. Este trabalho propõe um enquadramento baseado em estruturas de agregação hierárquica, mantendo o detetor fixo. Uma vista global única ou uma monitorização totalmente desagregada é frequentemente inadequada: uma vista global pode ocultar anomalias locais, enquanto a desagregação total é dispendiosa e pode deixar muitas séries demasiado esparsas para uma deteção fiável. Em vez disso, este trabalho explora estruturas hierárquicas parciais que podem revelar padrões anómalos de tráfego a um nível de detalhe relevante, com o objetivo de detetar anomalias em conjuntos de dados agrupados complexos sem sacrificar a interpretabilidade nem incorrer num custo analítico proibitivo.

Uma gramática formal gera automaticamente estruturas hierárquicas. Cada candidata é pontuada com o Matrix Profile, e a área sob a curva ROC (AUC) dessas pontuações é a aptidão da procura. Para além de hierarquias predefinidas, o método pode descobrir automaticamente estruturas de agregação úteis. Para este efeito, recorre-se à Evolução Gramatical (GE) (O'Neill and Conor Ryan, 2001; O'Neill and C. Ryan, 2003). A principal contribuição é um enquadramento reproduzível e interpretável para a procura de estruturas de agregação com um detetor fixo em séries temporais de telecomunicações. As hierarquias evoluídas atingiram uma AUC de teste de $0{,}678 \pm 0{,}066$, cerca de 0,10 a 0,14 acima da série totalmente agregada e das divisões por Call, Event e Service Type ($p = 0{,}002$).

Palavras-chave: deteção de anomalias, telecomunicações, séries temporais hierárquicas, séries temporais agrupadas, Evolução Gramatical, Matrix Profile.

Contents

Abstract . . . 5
Resumo . . . 6
List of abbreviations and acronyms . . . 9
List of Figures . . . 10
List of Tables . . . 11

1 Introduction **12**
1.1 Motivation . . . 12
1.2 Objectives . . . 12
1.3 Contributions . . . 13
1.4 Document Organization . . . 13

2 Background **14**
2.1 Search Method . . . 14
2.2 Time Series . . . 14
2.3 Anomaly Detection . . . 17
2.4 Anomaly Generation . . . 18
2.5 Grammatical Evolution . . . 18
2.6 Metrics . . . 21
2.7 Literature Review . . . 23

3 Materials and Methodology **27**
3.1 Problem Formulation . . . 27
3.2 Adopted Methodology . . . 31
3.3 Anomaly Injection . . . 33
3.4 Hierarchical Data Structures . . . 37
3.5 Fitness Evaluation . . . 40
3.6 Genetic Programming Implementation . . . 41
3.7 Technological Tools and Computational Environment . . . 43

4 Results and Discussion **47**
4.1 Experimental Setup . . . 47
4.2 Performance Analysis . . . 49
4.3 Time Series Visualisation . . . 52
4.4 Genetic Programming Analysis . . . 53
4.5 Discussion . . . 57
4.6 Limitations . . . 59

5 Conclusions and Future Works **60**
5.1 Conclusions 60
5.2 Future Work 61

List of abbreviations and acronyms

AUC *Area Under the ROC Curve*

BNF *Backus–Naur Form*

CDR *Call Detail Record*

CNN *Convolutional Neural Network*

GA *Genetic Algorithm*

GE *Grammatical Evolution*

GP *Genetic Programming*

ISP *Internet Service Provider*

KPI *Key Performance Indicator*

LSTM *Long Short-Term Memory*

MAE *Mean Absolute Error*

MP *Matrix Profile*

PIG *Position-Independent Grow*

ROC *Receiver Operating Characteristic*

ROI *Region of Interest*

VAE *Variational Autoencoder*

List of Figures

2.1 A two-level hierarchical tree diagram 15
2.2 Australian domestic overnight trips, 1998 Q1–2017 Q4, by purpose and state 17
2.3 Generational loop in an evolutionary algorithm 19

3.1 Telco network traffic metrics (#Events and Event Duration) over July-October 2022 aggregated. 28
3.2 Daily #Events from 18–28 July 2022 29
3.3 Anomaly detection workflow 32
3.4 Anomaly injection cycle 34
3.5 Anomaly injection procedure 35
3.6 Log of the injected anomalies 36
3.7 Injected anomalies on their segment and on the total traffic 37
3.8 Fitness evaluation of a candidate hierarchy 41
3.9 PonyGE2 control flow diagram. 42

4.1 Training AUC of evolved hierarchies and fixed baselines 50
4.2 Test AUC of evolved hierarchies and fixed baselines 50
4.3 Original vs pooled test AUC 51
4.4 Test AUC against number of leaf series 52
4.5 Train vs test AUC 53
4.6 Leaf series, labelled anomalies and Matrix Profile scores 54
4.7 Fitness evolution 55
4.8 Best evolved hierarchy (seed 7) 56

List of Tables

3.1 Categorical features of the switch records 28
3.2 Cardinality and Dominant Categories in Telco Data 29
3.3 Information content by variable 30
3.4 Key Configuration Parameters (experimental campaign) 44
3.5 Technological tools employed 45

4.1 Configuration of the experimental campaign 48
4.2 ROC-AUC of evolved hierarchies and fixed baselines (original metric) . . . 49
4.3 Pooled ROC-AUC and threshold metrics on the test period 51
4.4 Structure of the best evolved hierarchy per run 57

Chapter 1

Introduction

1.1 Motivation

Unexpected changes in ordinary traffic patterns can occur at any instant and may result in network faults or quality-of-service problems. Telecom traffic is not a single univariate series; it can be viewed under several categorical dimensions. For example, a call can have different types, an event type may take many values, and a subscriber may belong to a given type. Traffic type, service type, partner-related information, and related categories create many candidate aggregations on which anomaly detection can be run. Expert labels for real incidents are scarce, so candidate aggregations cannot be ranked with standard supervised metrics. This dissertation focuses on injecting synthetic anomalies into each candidate aggregation and using *Area Under the ROC Curve* (AUC) on Matrix Profile discord scores as a label-free fitness signal.

Having a fully aggregated view can hide local anomalies, while monitoring every categorical combination separately is costly computationally and in terms of interpretability. Moreover, some fully disaggregated series contain so few events that anomalies cannot be identified with confidence. This creates a dilemma: which aggregation level should anomaly detection work on? Operators may use a fixed hierarchy or manual rules; for example, aggregating by protocol or subscriber type can make some anomalies visible. Running anomaly detection independently on every variable combination, however, remains expensive and hard to interpret. Therefore, automatic search methods are needed to identify suitable candidate aggregations.

1.2 Objectives

This dissertation aims to develop a framework for anomaly detection in grouped telecommunications time series with hierarchical aggregation, using Grammatical Evolution to evolve programmes from a formal grammar. From a data perspective, the aim is to work with the categorical dimensions of real mobile-network traffic records—call type, event type, subscriber type, traffic type, service type, partner-related information, and related fields—so that anomalies can be studied at global and local aggregation levels.

From a methodological point of view, aggregation choice is framed as a combinatorial search over those categorical dimensions. Candidate hierarchies are encoded in a *Backus–Naur Form* (BNF) grammar so that only valid aggregation structures are generated. Grammatical Evolution (PonyGE2) then evolves aggregation programmes from

that grammar. For each candidate, an aggregated series is built and scored with Matrix Profile (STUMPY) discord distances in a label-free way. Because anomaly labels are scarce, synthetic anomalies provide the evaluation signal: AUC on the discord scores is used as fitness, and evolved hierarchies are compared with fixed aggregation baselines.

The work is guided by three research questions, which are answered in Chapter 4:

1. Can the choice of aggregation be framed as a combinatorial search over a grammar, so that every candidate is a valid hierarchy?

2. Can a label-free Matrix Profile scorer, evaluated on synthetic anomalies, drive this search?

3. Do the evolved hierarchies detect anomalies better than fixed aggregations, such as the fully aggregated series, single-feature splits and full disaggregation?

1.3 Contributions

The main contributions of this dissertation are:

- the formulation of aggregation choice for anomaly detection in grouped telecommunications series as a grammar-based search, with a fitness built from a label-free detector and synthetic anomalies;

- an implementation in Python on top of PonyGE2 and STUMPY that evaluates candidate hierarchies on real operator data and can be reproduced from the saved runs;

- an empirical comparison of evolved and fixed hierarchies on a mobile network switch. The evolved hierarchies reached a test AUC of 0.678 ± 0.066 and were significantly better than the fully aggregated series and the Call, Event and Service Type splits, but not better than the best single-feature splits;

- evidence that conclusions about the best aggregation depend strongly on how false alarms are counted, since the event-weighted fitness ignores leaves without anomalies and can be exploited by the search.

1.4 Document Organization

The dissertation is organised into five chapters. Chapter 1 presents the motivation, objectives, research questions, contributions and document organisation. Chapter 2 is the background: the bibliographic search, the theory, and the literature review. Chapter 3 describes the telecommunications dataset, preprocessing and aggregation, synthetic anomaly injection, hierarchy construction, and the PonyGE2 implementation. Chapter 4 presents the experimental results, comparing aggregation strategies and examining the evolved hierarchical structures. Chapter 5 summarises the contributions and limitations of the present work, and outlines future work, including the use of real incident labels and online detection.

Chapter 2

Background

2.1 Search Method

To support this dissertation, a layered bibliographic search was used. The first step built a synonym and subtopic list covering grouped and hierarchical time series, telecom anomaly detection, Matrix Profile, combinatorial optimisation, Grammatical Evolution, and anomaly injection, and ran Boolean queries in Google Scholar. The queries focused on keywords such as "Multivariate Time Series", "Grouped time series", "Hierarchical Time Series Forecasting", "Hierarchical anomaly detection", "Time Series and Anomaly Detection in Telecommunication", "Combinatorial Optimization", "Anomaly injection", and "Matrix Profile".

Because the combined topic is sparse and hard to retrieve by keywords alone, LLM-based systems were also used to surface additional candidate papers and related terms. From a small set of highly relevant seed papers, "Cited by" and related-work links were followed to catch papers the synonym list missed.

Candidate articles were screened primarily by relevance to the research gap (telecom anomaly detection, aggregation and hierarchy, Matrix Profile and discord detection, combinatorial or evolutionary search, and Grammatical Evolution). Publication date (preferring work after 2015 where appropriate), venue quality, and citation impact were used as secondary signals, not as a fixed citation cut-off. Core databases for this stage were IEEE Xplore, Google Scholar, SpringerLink, MDPI, and PLOS.

2.2 Time Series

A time series is a sequence of observations indexed in time order, typically collected at regular intervals (e.g., hourly or daily) (Box et al., 2015). Irregularly spaced data (e.g., event-driven recordings) can also be treated as time-series data in a broader sense. Methodological rigor requires consistency in measurement definitions so that observations remain comparable over time. Time-series data are univariate if they contain a single variable; multivariate series incorporate additional variables while preserving temporal alignment and relevance.

2.2.1 Hierarchical Time Series

Hierarchical time series are structured time-series datasets arranged into hierarchical relationships. Typical hierarchies include geographical areas (e.g., national → state → city) or product categories (e.g., product group → SKU, stock keeping unit).

Hierarchical time series exhibit multi-level aggregation, where lower-level components combine through arithmetic operations (e.g., summation) to reconstruct higher-level series (R. J. Hyndman and G. Athanasopoulos, 2021). For example, consider a geographic hierarchy:

- **Level 1 (National)**: Total sales in a country.
- **Level 2 (Regional)**: Sales in the *North* and *South* regions.
- **Level 3 (Local)**: Sales in cities (e.g., *North: City A + City B*; *South: City C + City D*).

The national series equals the sum of the regional series (National = North + South), and each regional series equals the sum of its constituent cities. This coherence ensures consistency across levels (Wickramasuriya et al., 2019).

As shown in Figure 2.1, hierarchical time series arrange temporal data into several aggregation levels. The framework consists of:

- **Root level**: the whole temporal sequence is represented by the aggregate series y_t for $t = 1, \ldots, T$;
- **Intermediate levels**: disaggregated elements (e.g., $y_{A,t}$ and $y_{B,t}$) that partition the root series;
- **Leaf level**: the most detailed constituents (e.g., $y_{AA,t}$, $y_{AB,t}$, $y_{AC,t}$ under node A; $y_{BA,t}$, $y_{BB,t}$ under node B).

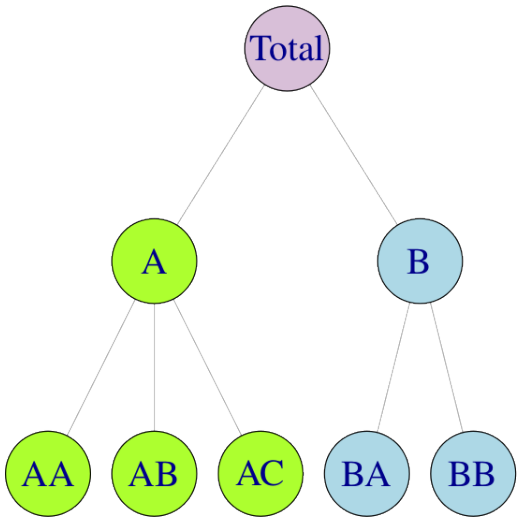


Figure 2.1: A two-level hierarchical tree diagram (R. J. Hyndman and G. Athanasopoulos, 2018).

Approaches to Hierarchical Forecasting

When analysing hierarchical time series, concurrent analysis at several aggregation levels can balance computational efficiency with granularity (R. J. Hyndman and G. Athanasopoulos, 2018).

Forecasting within a hierarchical system uses techniques to ensure that forecasts are credible and internally coherent: the aggregation of forecasts at one level should match the forecast of the level above. The main approaches are bottom-up, top-down, and middle-out (R. J. Hyndman and G. Athanasopoulos, 2018).

The bottom-up approach begins its predictions at the base level and aggregates them to higher levels. This approach is computationally practical and often performs well when upper-level series are relatively stable. The top-down approach starts at the top of the hierarchy and distributes forecasted totals using historical proportions. The middle-out approach connects bottom-up and top-down methods by focusing on a mid-level, aiming to balance the strengths of both.

2.2.2 Grouped Time Series

Grouped time series represent collections of temporally structured data organised by shared attributes (e.g., device type, customer demographics) rather than by a single nested hierarchy. Unlike hierarchical series, grouped data lack inherent aggregation constraints: forecasts for individual series may be developed independently and then analysed collectively for cross-sectional patterns (R. J. Hyndman and G. Athanasopoulos, 2018). Grouped time series are widely used in macroeconomic analysis (George Athanasopoulos et al., 2017) and supply-chain forecasting (Wickramasuriya et al., 2015), where multiple cross-sectional dimensions demand coherent alignment without a strict hierarchical dependence.

Often, disaggregating factors are both nested and crossed. For example, Australian tourism data can be disaggregated by hierarchical geographical categories (states, and regions nested within states), but also by four purposes of travel: holiday, business, visiting friends and relatives, and other. This grouping variable does not nest within any of the geographical variables.

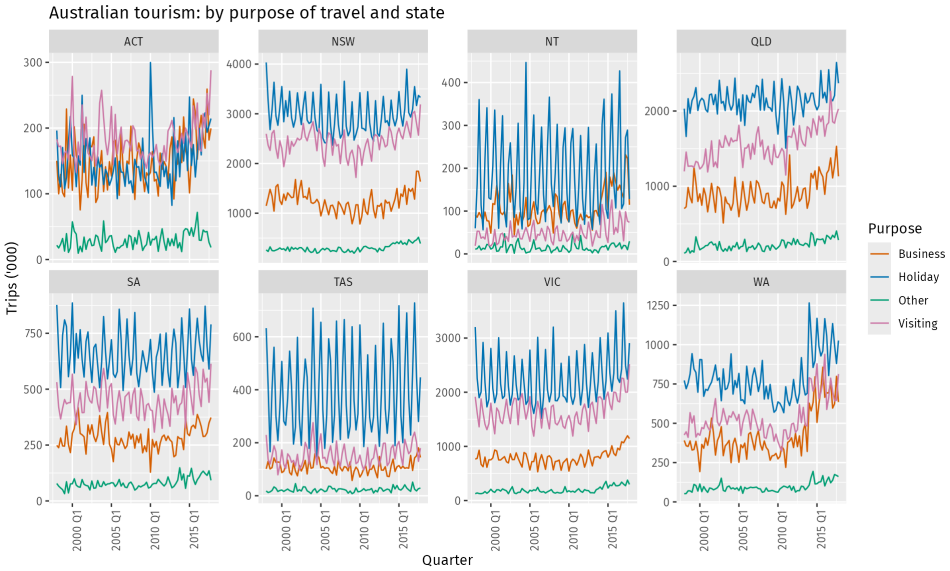


Figure 2.2: Australian domestic overnight trips over the period 1998 Q1 to 2017 Q4 disaggregated by purpose of travel and by state (R. J. Hyndman and G. Athanasopoulos, 2021).

2.3 Anomaly Detection

In machine learning, anomaly detection aims to find patterns that deviate significantly from expected behaviour (Chandola et al., 2009). It is essential in many fields, including fault diagnosis, network security, and fraud detection. Univariate anomaly detection identifies outliers in a single variable. Multivariate anomaly detection considers several variables jointly, so that a point may be anomalous because of an unusual combination of values even when each variable looks normal in isolation.

Anomalies are often classified as point anomalies (a single observation is unusual), collective anomalies (a contiguous subsequence is unusual as a group), or contextual anomalies (an observation is unusual only in a given context) (Chandola et al., 2009).

2.3.1 Matrix Profile and Discords

The Matrix Profile (*Matrix Profile* (MP)) is introduced here because Grammatical Evolution generates many aggregations, and for each produced candidate series a scorer is needed. MP is exact, label-light, and computationally feasible for many candidates. It takes one candidate time series and computes the nearest-neighbour distance for each subsequence of fixed length, then returns a discord score.

Distance is typically the z-normalised Euclidean distance between two subsequences (Yeh et al., 2017): each window is first standardised to zero mean and unit variance, so the comparison is based on shape rather than on the raw level of the series.

A time-series discord is the subsequence most distant from its nearest non-self match (Keogh et al., 2005). On the Matrix Profile, the discord is the position with the maximum nearest-neighbour distance (Yeh et al., 2017). For a short and unusual subsequence or spike, the discord may capture well, but may poorly capture a level shift, which is a sustained change to a new level; a ramp, which gradually drifts upwards over time; or a

multivariate contextual anomaly, where each series looks normal alone but the relationship is abnormal.

2.4 Anomaly Generation

Anomaly generation is the methodical production of irregular patterns or outliers that depart significantly from normal data distributions (Hawkins, 1980). In machine learning settings it is used both to stress-test system robustness (Chandola et al., 2009) and to augment training or evaluation data. Modern practice emphasises generating semantically plausible contextual anomalies for the target domain (Ruff et al., 2021).

2.4.1 Rule-Based Approaches

Expert-defined constraints in domain-specific anomaly generation deliberately break normal system behaviour (Chandola et al., 2009). In this dissertation, synthetic anomalies are injected with rule-based thresholds (e.g., values outside $\mu \pm 3\sigma$ over a 168-hour window).

2.4.2 Validation of Synthetic Anomalies

To ensure synthetic anomalies are useful for evaluation, validation typically combines domain checks (e.g., expert judgement that injected patterns are realistic) with quantitative detection benchmarks (Ruff et al., 2021). In this work, injected labels enable AUC evaluation of Matrix Profile discord scores on each candidate aggregation.

2.5 Grammatical Evolution

2.5.1 Combinatorial Optimization

Combinatorial optimisation identifies optimal configurations within finite discrete solution spaces and provides formal frameworks for many NP-hard problems (C. H. Papadimitriou and K. Steiglitz, 1982; Garey and Johnson, 1979). Exact methods (e.g., integer programming, branch-and-bound, dynamic programming) guarantee optimality but can be computationally costly at scale (Christos H Papadimitriou and Kenneth Steiglitz, 1998). Approximation and metaheuristic methods—including greedy search, local search, simulated annealing, and genetic algorithms—trade guarantees for tractable near-optimal solutions on large instances (Vazirani, 2001; Glover and Greenberg, 1989; Zlochin et al., 2004).

2.5.2 Genetic Programming

Genetic algorithms evolve a population of candidate solutions by selection, crossover, and mutation. Genetic programming (*Genetic Programming* (GP)) is a specialised form of genetic algorithm that evolves computer programmes, typically represented as trees whose nodes are functions or terminals (Koza, 1992). An operator of selection uses fitness to choose parents; crossover mixes elements of different parent solutions to produce offspring;

mutation introduces random variation; and a survival or replacement operator forms the next generation until a stopping criterion is met.

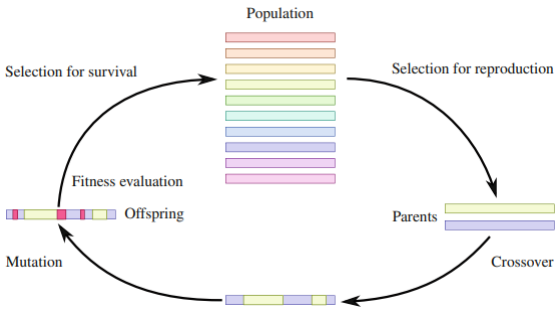


Figure 2.3: Generational loop in an evolutionary algorithm. From a population of solutions, individuals are selected to reproduce by crossover and mutation; offspring are evaluated and incorporated; and a survival operator restores the population size. Source: adapted from standard evolutionary-algorithm schematics (see also Koza (1992)).

GP is particularly effective for symbolic regression, automated algorithm discovery, and optimisation tasks (Koza, 1992; Banzhaf et al., 1998). Population size and the number of generations are crucial parameters that influence exploration of the search space and convergence speed.

Grammatical Evolution

Grammatical Evolution (GE) is a way to apply evolutionary computing to problems defined by a formal grammar (O'Neill and Conor Ryan, 2001; O'Neill and C. Ryan, 2003). A formal grammar is a set of rules that define a language's syntax and allow generation of valid strings or structures. In GE, the genotype is an integer vector rather than a tree; the grammar maps those vectors to valid phenotypes. This representation supports flexible adaptation to different grammar structures.

The process of Grammatical Evolution involves the following steps:

1. Define a BNF representation of the grammar;
2. Randomly generate genotypes as integer vectors;
3. Evaluate fitness by mapping genotypes to phenotypes and testing them against the fitness function;
4. Use genetic operators such as crossover and mutation to evolve the population of genotypes;
5. Repeat steps 3 and 4 until a satisfactory solution is found or a stopping criterion is met.

2.5.3 Backus-Naur Form

A BNF grammar describes the syntax of a language as a tuple of four sets:

- T: the terminal set;

- N: the non-terminal set;
- P: the set of production rules;
- S: the start symbol (a member of N).

For example, consider a simple grammar for mathematical expressions with addition, subtraction, multiplication, and division, using variables x and y:

$$T = \{+, -, /, *, x, y\}, \qquad N = \{\langle e\rangle, \langle o\rangle, \langle v\rangle\}.$$

The non-terminal $\langle e\rangle$ denotes an expression, $\langle o\rangle$ an operator, and $\langle v\rangle$ a variable. The production rules in P are:

$$\langle e\rangle ::= \langle e\rangle\langle o\rangle\langle e\rangle \mid \langle v\rangle \tag{2.1}$$
$$\langle o\rangle ::= + \mid - \mid / \mid * \tag{2.2}$$
$$\langle v\rangle ::= x \mid y \tag{2.3}$$

The pipe operator | denotes alternatives. The start symbol is $S = \langle e\rangle$.

Genotypes may be represented as non-negative integer arrays. For example, consider

$$C = [4, 15, 75, 8, 41, 12].$$

Mapping begins from the start symbol $\langle e\rangle$. The general rule is

$$\text{selected_option} = \text{gene_value} \bmod r,$$

where gene_value is the current integer from the genotype, r is the number of production choices available for the current non-terminal, and selected_option is the (zero-based) index of the chosen production.

The first gene is 4 and $\langle e\rangle$ has two options, so $4 \bmod 2 = 0$ selects $\langle e\rangle\langle o\rangle\langle e\rangle$. Continuing left-to-right:

1. With gene 15, $15 \bmod 2 = 1$ replaces the first $\langle e\rangle$ with $\langle v\rangle$;
2. With gene 75, $75 \bmod 2 = 1$ replaces $\langle v\rangle$ with y;
3. With gene 8, $8 \bmod 4 = 0$ replaces $\langle o\rangle$ with $+$;
4. With gene 41, $41 \bmod 2 = 1$ replaces the remaining $\langle e\rangle$ with $\langle v\rangle$;
5. With gene 12, $12 \bmod 2 = 0$ replaces $\langle v\rangle$ with x.

The final expression is $y + x$.

2.5.4 Initialization Phase

Initialisation can be performed on the genome or on the derivation tree. To control tree depth, this work uses Position-Independent Grow (*Position-Independent Grow* (PIG)), which generates individuals that tend to fill the allowed depth range: neither systematically too deep nor too shallow. PIG expands non-terminals in random order rather than always expanding leftmost first, and ensures that at least one branch reaches the maximum depth. This reduces duplicate structures and increases diversity in the initial population.

2.5.5 Selection Mechanism

The selection mechanism chooses individuals from the population to act as parents for the next generation while retaining some randomness to preserve diversity. This work adopts tournament selection: a number of individuals are drawn at random, the best individual in each tournament is retained, and the process is repeated until the required number of parents has been selected.

2.5.6 Variation Operators

The variation operators explore the search space by modifying genetic material. The two primary operators adopted in this work are crossover and mutation.

The crossover operator mixes parent genotypes to obtain offspring genotypes. Variable one-point crossover, which was chosen for this work, swaps the tails of two parents after cut points selected independently in each parent; because the cut positions need not coincide, offspring genome lengths may differ from those of the parents, enabling combination of beneficial traits of different lengths.

Mutation is applied to every child after crossover, using a codon-based integer-flip method. Here a codon is one element of the genotype vector: an integer flip replaces that codon with another random integer in the allowed codon range. Each codon is mutated independently with probability

$$P_{\text{mutation}} = \frac{1}{\text{genome_length}}. \tag{2.4}$$

In standard PonyGE2 practice this probability is applied over the genome (or over used codons, depending on configuration); the operator itself does not "know" which unused trailing genes will later be consumed by wrapping or unused material.

2.5.7 Replacement Strategy

The replacement strategy determines how new individuals enter the population. This work uses generational replacement with elitism: the best individuals from the previous generation are retained, while the remaining positions are filled by newly generated offspring. Elitism helps preserve high-quality solutions while the rest of the population continues to explore.

2.6 Metrics

2.6.1 Precision and Recall

In anomaly detection, precision and recall provide complementary perspectives on model performance, especially under class imbalance (e.g., rare fraud cases among many legitimate events).

Precision

Precision quantifies a model's ability to minimise false-positive classifications:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \tag{2.5}$$

where TP denotes true positives and FP false positives.

Recall

Recall evaluates how completely positive cases are identified:

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \tag{2.6}$$

where FN denotes false negatives.

2.6.2 Trade-off and Applications

Raising the decision threshold typically increases precision and decreases recall, and vice versa; the preferred balance is domain-specific. High recall is paramount in medical diagnostics, where missing a true positive can be severe. High precision is crucial in spam filtering, where false positives waste user attention.

F1 Score

The F1 score is the harmonic mean of precision and recall:

$$\text{F1} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}. \tag{2.7}$$

It provides a single score when precision and recall should be balanced, but can be misleading under severe class imbalance, where AUC or precision–recall curves may be more appropriate.

2.6.3 Area Under the ROC Curve (AUC)

The *Receiver Operating Characteristic* (ROC) curve plots the true-positive rate against the false-positive rate across classification thresholds (Hanley and McNeil, 1982). The Area Under the ROC Curve (AUC) is a common performance metric for binary classifiers under class imbalance (Fawcett, 2006). It measures the classifier's capacity to discriminate between classes.

Interpretation and Calculation

AUC values range from 0 to 1, where:

- 0.5 indicates random guessing;
- 1 represents perfect separability;
- values below 0.5 suggest inverted ranking.

The Wilcoxon–Mann–Whitney statistic provides a non-parametric AUC estimate (Hanley and McNeil, 1982):

$$\text{AUC} = \frac{\sum_{i=1}^{n_+} \sum_{j=1}^{n_-} \mathbf{1}(s_i^+ > s_j^-)}{n_+ \cdot n_-}, \tag{2.8}$$

where n_+ and n_- denote the numbers of positive and negative instances and s denotes classifier scores.

Advantages and Limitations

AUC is robust to class imbalance and does not require a fixed threshold (Bradley, 1997). However, it does not capture cost-sensitive decision contexts and can be misleading when ROC curves cross (Lobo et al., 2008).

2.7 Literature Review

2.7.1 Anomaly Detection in Telecommunications Networks

Telecommunication anomaly detection is difficult because labels are scarce. Moreover, the industry produces large amounts of data, with few viable labels that can be identified as anomalies. Often, the problem is not detecting the anomaly but selecting which subsection of the network window to monitor.

Gonzalez et al. (2024) apply a Dilated Convolutional Variational Autoencoder (DC-*Variational Autoencoder* (VAE)), a deep model that scores anomalies jointly across several related time series, to mobile-network counters. They evaluate it on a fully aggregated view where labels are sparse: of 730,000 samples, only 9,000 were labeled, using a fixed set of 12 features. Range-based F1 (range-F1) measures overlap between predicted and labeled anomaly intervals rather than isolated timestamps; they report a mean range-F1 of around 52%, which is only modest.

Burgueno et al. (2020) indicate that the network they analyzed had 25,000 mobile base station coverage areas. Moreover, for each cell they analyzed 240 hourly *Key Performance Indicator* (KPI)s, but the study focuses on only 15 of the most important. Detection is an online threshold state machine (normal / border / anomalous) run at cell level on that manually selected subset; they report capturing almost 80% of the labeled network anomalies. From this perspective, manual selection is a bottleneck because the 225 unmonitored features may still contain relevant information; for example, anomalies may occur in an unmonitored signal.

Call Detail Records (*Call Detail Record* (CDR)) and mobile activity records are recognized for analysis in telecom monitoring. Sultan et al. (2018) report no detection metrics.

Lemaire et al. (2024) study early detection of urban events from Orange France mobile Network Signalling Data in Paris: cell-level, one-minute counts of network services (calls, SMS, and related signalling). They use a Signature detector and hand-designed categorical aggregations over these service types, not temporal aggregation. Against a curated set of documented urban events, their best precision is 28.63%, with event recall of about 36% at the corresponding operating point, indicating many false alarms even when the signal is useful.

Existing work assumes a fixed aggregation and therefore may miss anomalies that would only appear under a different grouping. Automated search is useful when the best aggregation is not obvious in advance; in those papers, there is a fixed review of aggregation, but they do not address how to automate the aggregation of time series. The next section will be about this topic.

2.7.2 Anomaly Detection in Grouped Time Series

Anomaly detection in grouped time series treats the detector input as a chosen representation, not the whole network. The dataset is aggregated by related features/entities at

the aggregation level, rather than treating a network as a flat collection of features. It is important because the detection quality is dependent on the aggregation of features, not only the detector. Across the reviewed studies, datasets, labels, and metrics differ, so reported scores are not directly comparable.

Mefleh et al. (2025) used an unsupervised Mahalanobis distance approach in a telecommunications setting. They report AUC rising from 0.77 on the raw KPI features to 0.86 when denominator information is included (for example, attempt counts that normalise ratios), then to 0.93 with adjusted ratios, and finally to 0.99 with learned adjustments. This result shows that detection quality changes with feature transformation and aggregation.

Hua et al. (2022) show that China Mobile wireless base stations share some operational behavior, which GenAD, a generative reconstruction model, can learn and then use to detect anomalies on other stations in the same network. GenAD was pre-trained on 3,000 unlabeled base stations, using multivariate time series features. Their F1 score is 0.953 (precision 0.910). That F1 uses point-adjusted segment scoring: if any timestamp inside a labeled anomaly interval is flagged, the whole interval counts as detected, so the score is not a timestamp-wise F1.

In the study by Oguz and Kalaycioglu (2022), they simulate a multi-tier network that includes aggregations from the user equipment measurements and classifies the state of each femtocell access point. Performance changes with the region of interest (*Region of Interest* (ROI)) radius, signal shadowing, and aggregation methods. At a 30 m ROI with 2 dB shadowing, the 1D *Convolutional Neural Network* (CNN) achieved 80.1% accuracy, but as ROI size and the shadowing increase, the accuracy decreases, so this shows that including more observations is not necessarily beneficial when added signals are less relevant or noisier.

Based on the reviewed studies, we can extract that grouping can strongly affect anomaly detection performance. Although aggregations are selected manually before modeling the relevant KPIs, entities, aggregation levels, and spatial boundaries, none of them systematically search for an aggregation structure that best characterizes the anomaly.

2.7.3 Anomaly Detection in Hierarchical Time Series

Hierarchy in anomaly detection is important because an alert such as "traffic is abnormal" is not enough for a telecom operator to act on. The operator must determine whether it affects the entire network, a region or specific cell of the switch, or only one KPI category. So this top-down approach helps break down into smaller branches; although that hierarchy is not necessarily the best aggregation for detecting anomalies.

Hong et al. (2012) detect and localize anomalies in hierarchical *Internet Service Provider* (ISP) call-detail data. It analyzes customer-care call logs and signaling call records across predefined hierarchies, allowing operators to identify not only that an abnormal behavior occurs but also the level at which it appears. They found over 94% of reference anomalies, and separately reported 99.7% anomaly identification accuracy. These results were obtained from a fixed additive hierarchy and Holt–Winters spike forecasting. So the author demonstrates hierarchical localization rather than searching over alternative aggregation structures. It does not search for alternative partial aggregations or score non-spike discords, which is the gap addressed by this thesis.

On the other hand, Wolff et al. (2022) treat anomaly localization as the sequential

search problem of a known tree of processes. Instead of measuring every leaf process equally, their hierarchical dynamic search method first samples aggregate internal nodes and then moves towards the branch most likely to contain an anomaly. This shows that hierarchy can be an active search space, not merely a reporting structure used after detection. However, Wolff assumed tree topology is already known, as search is only within it. This differs from our approach, which uses a GE to generate candidate aggregations before applying the anomaly scorer.

Huang et al. (2026) propose HYDRA, a multi-level hierarchy-driven anomaly detector that builds several hierarchy-based views of each series and combines detection signals across those levels. It ranked first among 40 algorithms across 40 TSB-AD datasets.

2.7.4 Matrix Profile and Time-Series Discord Detection

Chapter 2 defined the Matrix Profile and discords. What belongs in this chapter is the published evidence that the scorer is fast enough for many candidate series, and whether it has been used on telecommunications data.

Computational efficiency is substantially improved: a refinery time-series self-join is reduced from an estimate of 153.8 days by brute force to 1.2 hours with STAMP/STOMP (Yeh et al., 2017). Also, Zhu et al. (2016) showed GPU-STOMP could process 100 million subsequences in about 12 days, compared to the estimate of 1585 years for brute force. STUMPY (Law, 2019) gives the practical implementation of this process in Python. Van Onsem et al. (2023) apply Matrix Profile to production KPI series from telecom-provider customers of Skyline Communications (25 series), using it for variable-length *motif* discovery rather than discord scoring. They report the method is about 3–8 times faster than VALMOD; an example series yields both a 288-point (about one day) motif and a 12-point (about one hour) motif in a single distance-matrix pass. No detection precision, recall, F1, or AUC is reported (manual motif checks). The series to analyse are chosen in advance, so the paper does not search aggregations.

2.7.5 Combinatorial Optimisation for Selecting Time Series

Little published work searches hierarchical aggregation structures. Feature-selection methods are reviewed here instead because, from an optimisation viewpoint, the problems are similar: both search a combinatorial space of candidates and score each one with a fitness function. The spaces themselves differ: feature selection chooses a subset of columns, whereas this dissertation searches grammar-constrained hierarchies.

For feature selection, n features give 2^n candidate subsets, so exhaustive search is infeasible for large n; Xue et al. (2016) survey the evolutionary algorithms used for this search, including genetic algorithms, genetic programming, particle swarm optimisation, ant colony optimisation and differential evolution.

Linares-Barrera et al. (2024) use a genetic algorithm (SEISEN) whose fitness is a filter score, the Conditional Dependence Coefficient, to select lagged features for a multilayer perceptron forecaster. On an electricity dataset, the feature set falls from 7,704 to 3,813 while *Mean Absolute Error* (MAE) improves from 207.193 to 194.400, showing that pruning the input representation can improve accuracy.

Pietroń et al. (2025) apply a genetic algorithm at three levels to multivariate time-series anomaly detection: it evolves subsets of input series, an autoencoder architecture for each subset, and the network weights, using validation loss as fitness. On four sensor

benchmarks (SWaT, WADI, MSL and SMAP), the resulting ensemble achieves the best point-wise F1 of 14 methods, with a mean of 0.70 against 0.54 for the next best, at a cost of up to 132 GPU-hours per dataset.

He et al. (2021) search subspaces of high-dimensional financial transaction series with a greedy heuristic and an evolutionary $(\mu + \lambda)$ algorithm, scoring each candidate by discord mining. On real data, greedy search achieves the higher mean average precision (0.86 against 0.72) but is about 15 times slower (1,345 s against 88 s).

Across these studies, the common pattern is an evolutionary search over a large candidate space with a fitness function evaluated per candidate. None searches hierarchical aggregation structures, and none combines the search with a Matrix Profile scorer on telecommunications data; this dissertation does both with grammatical evolution.

Chapter 3

Materials and Methodology

This chapter describes the data, the method and its implementation. Section 3.1 presents the telecommunications dataset, analyses its categorical features and states the problem formally. Section 3.2 gives an overview of the pipeline. The following sections describe each stage in detail: the injection of synthetic anomalies (Section 3.3), the representation of candidate hierarchies with a grammar (Section 3.4), the fitness function that scores a hierarchy with the MP and the AUC (Section 3.5), and the configuration of the GE search (Section 3.6). Section 3.7 closes the chapter with the software tools and the computational environment.

3.1 Problem Formulation

This research develops a hierarchical anomaly detection framework for mobile network data, with the aim of improving service reliability. Its central question is at which level of aggregation the traffic should be monitored: a single fully aggregated series hides anomalies that affect a small segment of the traffic, while one series per combination of categorical values produces too many sparse series to monitor and interpret. The framework answers this question with a search: candidate aggregation hierarchies are generated by a grammar, evolved with GE, and ranked by how well a label-free detector finds known anomalies in the series they produce.

3.1.1 Data Description

The data comes from a network switch of a mobile operator and covers July to October 2022. Each record counts the call events of one combination of categorical values in one hour, and corresponds to an aggregate of CDRs. The raw extract is a semicolon-separated file in which date and hour are stored in separate columns. During preparation the two are combined into one hourly timestamp, the #Events column is cleaned of non-numeric characters and converted to integers, the columns not used in this work (data source, date, hour and Event Duration) are dropped, and duplicated records are removed. The resulting working dataset of 958,479 hourly records, spanning 2,110 hours from 18 July to 13 October 2022, is used throughout this dissertation.

The switch reports two metrics: #Events, the number of call events, and Event Duration, the total call time. Only #Events is used as the target variable in this work, because event counts are available for both voice and SMS traffic and are the usual indicator of traffic volume. Each record is described by six categorical features, listed in Table 3.1.

Table 3.1: Categorical features of the switch records. The letter is the symbol used for the feature in the grammar (Section 3.4.2).

Letter	Feature	Description and values
A	Call Type	Type of communication: SMS, VOICE.
B	Event Type	Role of the event in the call: ORIGINATED, CALL FORWARD, TERMINATED, ROAMING OUT.
C	Subscriber Type	Type of subscriber: Postpaid, Application, Roamer In, Prepaid, Other.
D	Traffic Type	Destination of the traffic: OFF-NET (another operator), INT (international), ON-NET (same operator).
E	Service Type	Whether the subscriber is roaming: NOT-ROAMING, ROAMING.
F	Partner Description	Anonymised partner operator associated with the traffic: 392 values, P_0001 to P_0392.

Figure 3.1 shows the fully aggregated traffic over the whole period and Figure 3.2 zooms into eleven days of #Events. Both show a strong daily cycle, with low traffic at night and peaks during the day, and a weekly pattern in which weekend days differ from working days. This regularity is what makes a shape-based detector suitable: an anomaly appears as an hour that breaks the expected daily and weekly shape. It also motivates the window of about one week used both to inject anomalies (Section 3.3) and to compute the MP (Section 3.5), since one week is the shortest window that contains a complete cycle of both patterns.

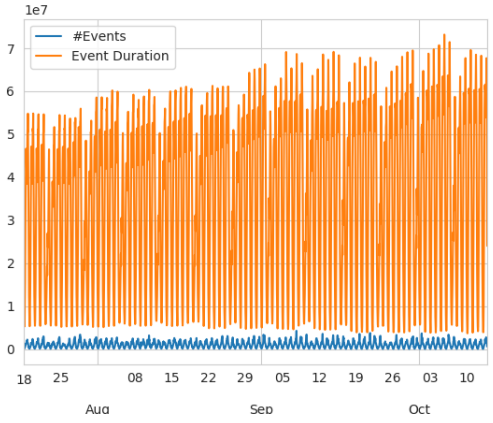


Figure 3.1: Hourly network traffic patterns showing (A) Total #Events (left axis) and Event Duration in hours (right axis).

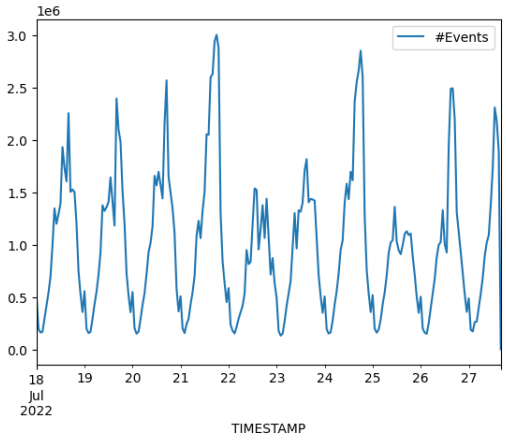


Figure 3.2: Hourly call event frequency.

3.1.2 Categorical Feature Analysis

The six features can in theory be combined into $2 \times 4 \times 5 \times 3 \times 2 \times 392 = 94{,}080$ distinct combinations, but only 1,555 of them occur in the data. Each non-empty combination is observed on average in 614 of the 2,110 hours of the period, with a median of only 182 hours: 94 combinations are present in every hour, while 127 appear in a single hour, so most of the corresponding series are zero for most of the time. Table 3.2 describes each feature over these 1,555 combinations, and adds the share of the total #Events carried by the most frequent category.

Table 3.2: Cardinality and Dominant Categories in Telco Data. Unique Values is the number of categories per variable; Frequency counts are out of the 1,555 non-empty feature combinations, not out of records. The last column is the share of all #Events carried by the same top category.

Variable	Unique Values	Top Category	Frequency	Share of #Events
Call Type	2	VOICE	849 (54.6%)	45.2%
Event Type	4	ORIGINATED	716 (46.0%)	18.3%
Subscriber Type	5	Roamer In	1,486 (95.6%)	5.7%
Traffic Type	3	INT	1,486 (95.6%)	21.8%
Service Type	2	NOT-ROAMING	1,544 (99.3%)	92.4%
Partner Description	392	P_0001	65 (4.2%)	94.3%

The table reveals three distribution patterns:

- **Highly skewed features.** Subscriber Type, Traffic Type and Service Type are dominated by one category in terms of combinations: 95.6% of the combinations are Roamer In and international (INT) traffic, and 99.3% are not roaming. Inbound roamers are associated with many different partner operators, so they generate most of the distinct combinations.
- **Balanced features.** Call Type (54.6% VOICE, 45.4% SMS) and Event Type (46.0% ORIGINATED) split the combinations more evenly.

- **High-cardinality feature.** Partner Description has 392 values with a long-tail distribution; the most frequent partner appears in only 4.2% of the combinations.

Counting combinations and counting traffic give very different pictures. Roamer In accounts for 95.6% of the combinations but only 5.7% of the events: most of the traffic comes from Prepaid subscribers (63.3% of all #Events) and stays inside the operator's network (ON-NET, 61.3%). Conversely, partner P_0001 appears in only 4.2% of the combinations but carries 94.3% of the events. In other words, a few combinations carry most of the traffic, while most combinations are small and sparse. This matters for the method: a leaf of a hierarchy that isolates a high-volume segment produces a dense, regular series, whereas a leaf that isolates a rare combination produces a sparse series that is hard to score (Section 3.5).

The information content of each feature is measured with the Shannon entropy of its distribution over the combinations,

$$H = -\sum_{i=1}^{n} p_i \log_2 p_i, \tag{3.1}$$

where p_i is the proportion of combinations with category i and n the number of categories. The entropy is 0 bits when a feature takes a single value and reaches $\log_2 n$ when all values are equally frequent. Table 3.3 lists the values.

Table 3.3: Information content by variable, computed over the 1,555 non-empty combinations. The maximum is the entropy of a uniform distribution over the same number of categories.

Variable	Entropy (bits)	Maximum $\log_2 n$ (bits)
Call Type	0.994	1.000
Event Type	1.361	2.000
Subscriber Type	0.349	2.322
Traffic Type	0.306	1.585
Service Type	0.061	1.000
Partner Description	8.404	8.615

Partner Description has by far the highest entropy (8.4 bits, close to its maximum), so splitting on it creates hundreds of branches, most of them containing very little traffic. This has two consequences. First, its 392 values would make the grammar and the search space much larger: every split on Partner Description would create 392 child nodes. Second, the resulting leaves would be the sparse series on which the detector performs worst. Partner Description is therefore excluded from the search space. It is present in the data and in the full grammar `grammars/mobai.pybnf` (letter F), but it is dropped from `grammars/mobai_nop.pybnf`, the grammar used in the experimental campaign. Without it, only 77 of the $2 \times 4 \times 5 \times 3 \times 2 = 240$ possible combinations of the remaining five features occur in the data. At the other extreme, Service Type has almost no entropy (0.06 bits): nearly all combinations are not roaming, so a split on it produces one large branch and one very small one. It is kept in the grammar, and the search decides whether such a split is useful.

3.1.3 Formal Problem Statement

Let each record r have values $x_1(r), \ldots, x_5(r)$ for the five features A–E, an hour $t(r)$, an event count $e(r)$ and an anomaly flag $a(r) \in \{0, 1\}$ (Section 3.3). A *filter* ϕ is a conjunction of conditions of the form $x_j = v$, with at most one condition per feature, for example $\phi =$ (Event Type = ORIGINATED) $\wedge$ (Call Type = SMS). Each filter defines an hourly series and its labels:

$$s_\phi(t) = \sum_{r\,:\,\phi(r),\ t(r)=t} e(r), \qquad y_\phi(t) = \max_{r\,:\,\phi(r),\ t(r)=t} a(r), \tag{3.2}$$

so $s_\phi(t)$ is the number of events of the matching records at hour t, and hour t is anomalous in this series if any of these records was modified by the injection. Hours without matching records have $s_\phi(t) = 0$ and $y_\phi(t) = 0$.

A *hierarchy* H is a tree whose root represents all traffic. Each internal node splits its traffic on one feature not yet used on the path from the root, creating one child per value of that feature; each leaf ℓ therefore corresponds to the filter ϕ_ℓ formed by the conditions on its path. Because every split covers all values of the feature, the leaves of a hierarchy partition the records: each record belongs to exactly one leaf, so no traffic is lost or counted twice. The fully aggregated series is the hierarchy with a single leaf and an empty filter.

Let $\mathcal{L}(G)$ be the set of hierarchies generated by the grammar G (Section 3.4) and $F(H; D)$ the fitness of hierarchy H on the data D, defined in Section 3.5. The problem is to find

$$H^* = \arg\max_{H \in \mathcal{L}(G)} F(H; D_{\text{train}}), \tag{3.3}$$

the hierarchy whose leaf series allow a fixed, label-free detector to separate the injected anomalies from normal hours as well as possible on the training period. The hierarchy found is then evaluated, without further changes, on the test period D_{test}.

3.2 Adopted Methodology

Figure 3.3 summarises the pipeline, which has five stages.

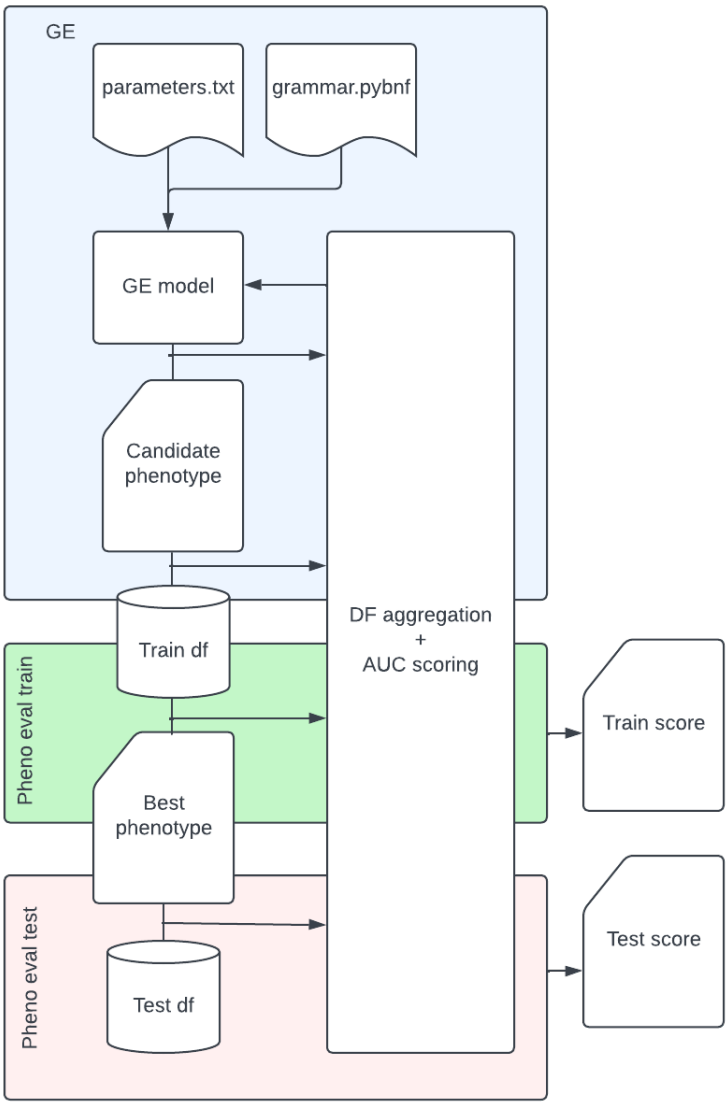


Figure 3.3: Hierarchical anomaly detection pipeline combining data aggregation, synthetic anomaly injection, and genetic model optimization.

1. **Data preparation.** The raw extract is cleaned, the unused columns are dropped and duplicated records are removed, leaving 958,479 hourly records with six categorical features and the #Events count. The records are kept disaggregated: series are built later, separately for each hierarchy, because each hierarchy defines its own aggregation.

2. **Anomaly injection.** The operational data has no reliable anomaly labels, so synthetic point anomalies are inserted into 105 randomly chosen (combination, hour) pairs, one for about 5% of the 2,110 hours, and the modified records are flagged (Section 3.3). These flags are the ground truth for the evaluation; the detector itself never sees them.

3. **Data partitioning.** The data is split temporally, without shuffling, so that the test period lies entirely after the training period, as it would in operation. The training period runs from 18 July to 14 September 2022 (1,411 hours, 630,861 records) and

the test period from 14 September to 13 October 2022 (699 hours, 327,618 records), roughly two thirds and one third of the data. The training period contains 75 anomalous hours and the test period 28.

4. **Hierarchy search.** PonyGE2 evolves a population of candidate hierarchies generated by the grammar `mobai_nop.pybnf` (Sections 3.4 and 3.6). Each candidate is evaluated on the training period only: its leaf series are built with Equation (3.2), scored with the MP, and compared with the anomaly flags through the weighted AUC (Section 3.5).

5. **Evaluation.** The best hierarchy of each run is re-evaluated on the training and test periods and compared with fixed aggregation baselines (Chapter 4).

The detector is the same in every case. What changes from one candidate to another is only the set of series it is applied to, so differences in fitness measure the effect of the aggregation and not of the detector.

3.3 Anomaly Injection

Since the operational dataset does not contain reliable anomaly labels, synthetic anomalies are injected to create a labelled evaluation scenario. Injection has two advantages over expert labels: the position of every anomaly is known exactly, and anomalies can be placed at every level of aggregation, from the whole network to a single combination of categories. The procedure is shown in Figures 3.4 and 3.5.

Each anomaly is created in four steps.

1. **Choice of the affected segment.** Each of the six categorical features is selected independently with probability 0.5, and a value is drawn at random for each selected feature, for example $\{\text{Service Type} = \text{ROAMING}, \text{Traffic Type} = \text{INT}\}$. The number of conditions therefore varies: few conditions give a broad segment, such as all international traffic, and many conditions a narrow one, such as one partner for one subscriber type. If no feature is selected the anomaly affects all traffic. A combination that was already used, or that matches no records, is discarded and a new one is drawn. All six features are eligible, including Partner Description, so some anomalies affect segments that the grammar cannot isolate.

2. **Choice of the hour.** An hour t^* is drawn uniformly among the hours in which the segment has records.

3. **Reference distribution.** The hourly #Events series of the segment is built over the hours in which it has records, and its mean μ and standard deviation σ are computed over a window of 167 observations around t^* (the 84 before it, t^* itself and the 82 after it, truncated at the ends of the series). For a segment present in every hour this window covers one week minus one hour and therefore a complete daily and weekly cycle; for a sparse segment, which has no records in some hours, the same number of observations spans a longer period. If the segment has a single observation, no window can be formed and the value is instead halved or increased by half, with equal probability.

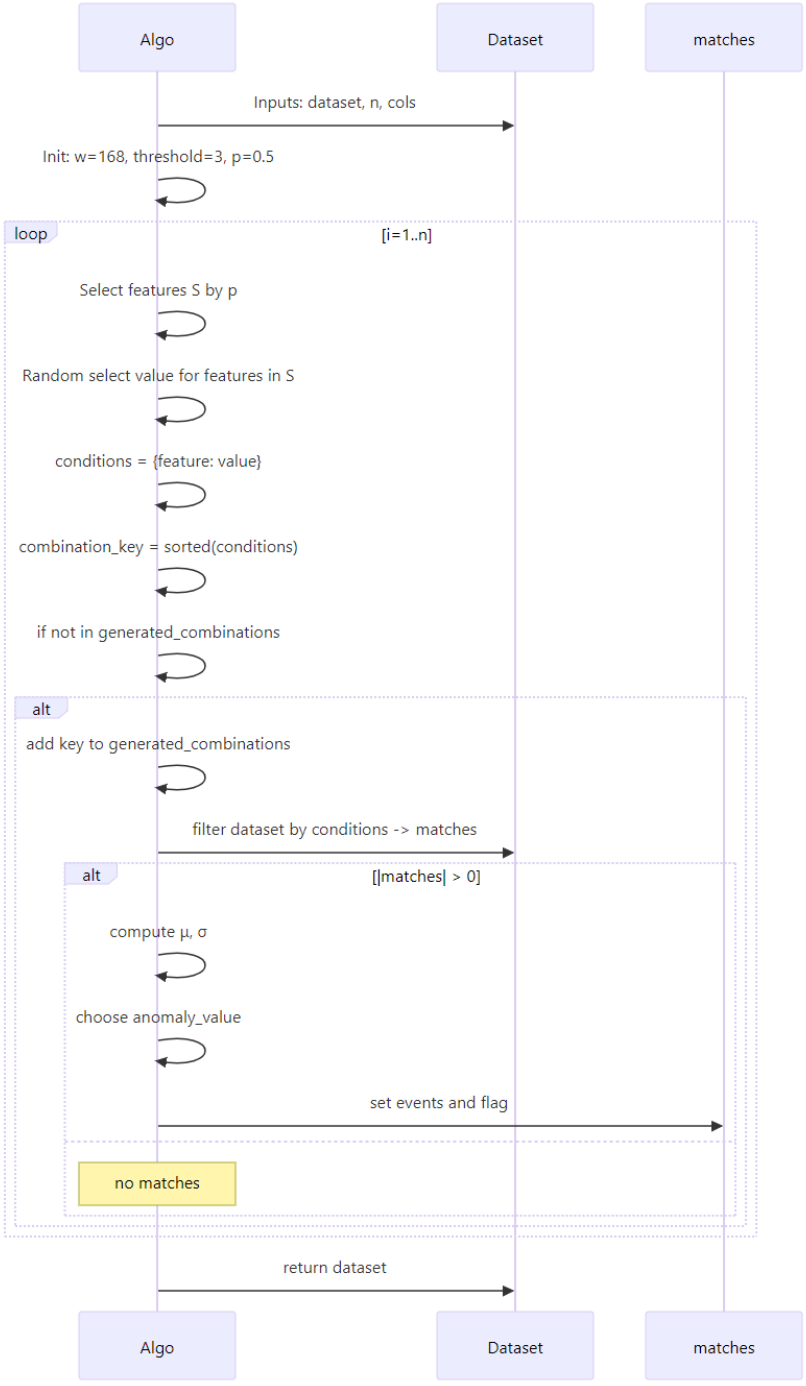


Figure 3.4: Anomaly injection cycle.

4. **Modification.** With equal probability, the anomaly is a peak or a drop. The new value of the segment at t^* is $v^* = \mu + 3\sigma$ for a peak and $v^* = \max(\mu - 3\sigma, 0)$ for a drop, since an event count cannot be negative. The records of the segment at t^* are rescaled by the same factor $v^*/s(t^*)$, where $s(t^*)$ is their original total, so the anomaly is distributed over the records in proportion to their original values; the new counts are rounded to integers and the records are flagged as anomalous.

Require: records D, features $X_1, \ldots, X_6$, number of anomalies n, window $w = 167$, $k = 3$

$U \leftarrow \emptyset$ ▷ combinations already used
while fewer than n anomalies have been inserted **do**
  $S \leftarrow \{X_j : \text{a uniform random number} < 0.5\}$
  $\phi \leftarrow \bigwedge_{X_j \in S}(X_j = v_j)$, with each v_j drawn from the values of X_j
  if $\phi \in U$ **or** no record matches ϕ **then continue**
  $U \leftarrow U \cup \{\phi\}$
  $t^* \leftarrow$ hour drawn uniformly among the hours with records matching ϕ
  $\mu, \sigma \leftarrow$ mean and standard deviation of s_ϕ over the w observations around t^*
  $v^* \leftarrow \mu + k\sigma$ or $\max(\mu - k\sigma, 0)$, each with probability 0.5
  for each record r with $\phi(r)$ and $t(r) = t^*$ **do**
    $e(r) \leftarrow \text{round}\big(e(r) \cdot v^*/s_\phi(t^*)\big)$; $a(r) \leftarrow 1$
return D

Figure 3.5: Anomaly injection procedure.

Figure 3.6 shows part of the log produced by the procedure. In the first row the segment is all international traffic (Traffic Type = INT) at 08:00 on 10 September 2022: with $\mu = 234{,}009$ and $\sigma = 119{,}169$, the peak value is $\mu + 3\sigma = 591{,}516$, against an actual value of 261,851, so every matching record is multiplied by about 2.26. Most of the other rows shown are drops. For hourly traffic σ is large relative to μ, because the daily cycle alternates busy and quiet hours, so $\mu - 3\sigma$ is negative and every drop sets the segment to zero. Drops therefore behave like outages of the segment: 30 of the 75 anomalous hours in the training period and 12 of the 28 in the test period are drops to zero, and the rest are peaks. The 105 anomalies fall on 103 distinct hours, because in two cases the hour drawn for a new segment had already received an anomaly on another segment.

The procedure was implemented in a Kaggle notebook and executed once, without a fixed random seed. Its output, the full record table with the modified counts and the anomaly flags, was saved and then split into the training and test sets, which are used unchanged in every run and in the final evaluation; the injected anomalies are therefore identical across all experiments, even though a new execution of the procedure would draw different ones.

The window of about one week only defines the reference distribution: each anomaly is a *point* anomaly that affects one hour of one segment, and the rest of the series is unchanged. An anomaly is extreme relative to *its own* segment, but it is diluted in any series that also contains other traffic. Figure 3.7 illustrates this with the two anomalies discussed above. The peak on international traffic more than doubles that segment, but in the total traffic, of which INT is only about a fifth, the same hour lies well inside the daily range. The drop sets CALL FORWARD events to zero, a clear outage of that

	TIMESTAMP	Mean	STD	Anomaly Value	Anomaly Value from DF	Actual Value	Used Conditions
0	2022-09-10 08:00:00	234009.095808	119169.051509	591516.250337	591523	261851	{'Traffic Type': 'INT'}
1	2022-08-01 21:00:00	72158.305389	43227.338864	0.000000	0	88799	{'Event Type': 'CALL FORWARD'}
2	2022-08-24 02:00:00	4093.964072	2066.479431	0.000000	0	744	{'Call Type': 'VOICE', 'Subscriber Type': 'Roa...
3	2022-10-06 11:00:00	94760.401198	61335.916119	0.000000	0	183307	{'Call Type': 'VOICE', 'Subscriber Type': 'Pos...
4	2022-08-07 12:00:00	45874.910180	32270.480622	142686.352045	142686	57631	{'Call Type': 'SMS', 'Subscriber Type': 'Postp...
...	...	...	...	...	...	...	...
100	2022-07-27 09:00:00	79092.904192	47411.075440	221326.130513	221326	127617	{'Call Type': 'VOICE', 'Subscriber Type': 'Pos...
101	2022-10-10 04:00:00	17.680000	23.986663	89.639989	90	14	{'Event Type': 'TERMINATED', 'Service Type': '...
102	2022-10-07 14:00:00	266.125749	558.001988	0.000000	0	615	{'Call Type': 'VOICE', 'Subscriber Type': 'App...
103	2022-07-27 10:00:00	202.976048	140.889043	0.000000	0	188	{'Partner Description': 'P_0092'}
104	2022-08-17 04:00:00	9796.377246	5771.764798	0.000000	0	1587	{'Subscriber Type': 'Prepaid', 'Traffic Type':...

Figure 3.6: Part of the log of injected anomalies: hour, mean and standard deviation of the segment over the reference window, target value v^*, value obtained after rounding, original value and conditions that define the segment.

segment, while the total traffic at that hour looks like an ordinary evening value. An anomaly is easiest to detect in a series that is close to its segment, which is precisely why the choice of aggregation matters and why it is the object of the search. Conversely, in a series that is much narrower than its segment the anomaly is spread over several leaves, each receiving the same relative change.

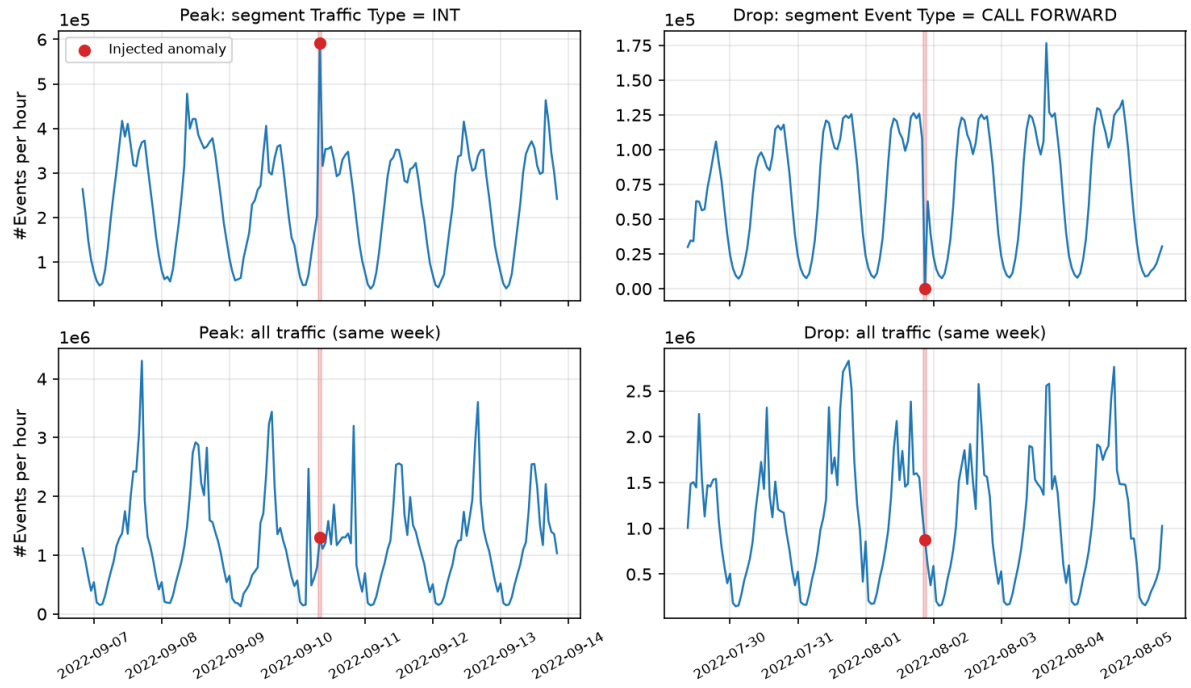


Figure 3.7: Two injected anomalies, a peak (left) and a drop (right), shown over one week on the segment that received them (top) and on the fully aggregated series (bottom). The anomalous hour is marked in red.

3.4 Hierarchical Data Structures

There is no unique way to aggregate the data. The same records can be monitored as one total series, as one series per call type, as one series per combination of call type and subscriber type, or by any mixture in which different branches are split on different features. This section describes how the space of admissible hierarchies is defined with a grammar and how a hierarchy is turned into the set of series that are scored.

3.4.1 Size of the Search Space

Let $T(S)$ be the number of distinct hierarchies that can be built below a node when the features in S are still unused on its path. The node is either a leaf, or it splits on some feature $f \in S$ into k_f children, each of which is again an arbitrary hierarchy over $S \setminus \{f\}$:

$$T(\emptyset) = 1, \qquad T(S) = 1 + \sum_{f \in S} T(S \setminus \{f\})^{k_f}. \tag{3.4}$$

With the five features A–E and $k_f = 2, 4, 5, 3, 2$ values, $T(\{A, \ldots, E\}) \approx 2.2 \times 10^{61}$. Exhaustive evaluation is therefore impossible, even though each hierarchy has at most 240 leaves, and the hierarchy must be found by a heuristic search. The number of hierarchies grows so quickly because the branches of a node are independent: the traffic of one call type may be split by subscriber type while the traffic of the other call type is not split at all.

3.4.2 Grammar

The admissible hierarchies are defined by the BNF grammar `grammars/mobai_nop.pybnf`, which has a single non-terminal `<node>`:

```
<node> ::= {:A0<node>{::}A1<node>:} |
           {:B0<node>{::}B1<node>{::}B2<node>{::}B3<node>:} |
           {:C0<node>{::}C1<node>{::}C2<node>{::}C3<node>{::}C4<node>:} |
           {:D0<node>{::}D1<node>{::}D2<node>:} |
           {:E0<node>{::}E1<node>:} |
           -END
```

Each of the first five productions splits the current node on one feature, and the last one, `-END`, makes it a leaf. A label such as `C2` combines a letter, which identifies the feature (Table 3.1), and a digit, which selects a value from that feature's list of categories in `datasets/Mobai/available_categories.json`; for example, `C2` is Subscriber Type = Roamer In. A split production lists *all* values of its feature, each followed by a new `<node>`, so every split creates one child per value and the leaves always partition the traffic of their parent. The symbols `{:`, `{::}` and `:}` are PonyGE2 formatting markers that open an indented block, start a new line at the same indentation and close the block. The phenotype is therefore an indented text in which indentation encodes depth in the hierarchy, as in this excerpt of the best hierarchy evolved in the campaign (Chapter 4):

```
B0
  A0-END
  A1
    C0
      C0-END
      ...
```

This fragment reads: split all traffic by Event Type; within ORIGINATED (`B0`), split by Call Type; SMS (`A0`) is a leaf; within VOICE (`A1`), split by Subscriber Type, and so on.

The grammar does not prevent a feature from being used twice on the same path, as in the second `C0` above. Such a split is meaningless, since all the records under `C0` already have that Subscriber Type, so it is resolved when the phenotype is parsed: the branch is truncated and its parent becomes a leaf. Keeping the grammar this simple has a cost, because part of the genetic material encodes truncated branches that do not affect the hierarchy, but it means that every derivation is a valid hierarchy and no repair or penalty is needed. The fully aggregated series is also in the language of the grammar: it is the phenotype `-END`, a root that is a leaf.

3.4.3 From Genotype to Hierarchy

As described in Chapter 2, the genotype is a list of integer codons, and each codon selects a production with the rule codon mod r, where r is the number of productions of the non-terminal being expanded. Here `<node>` is the only non-terminal and has $r = 6$ productions, numbered 0 (split on A) to 5 (`-END`), and the non-terminals are expanded depth first, from left to right. The genotype of the best hierarchy of the campaign starts with the codons 35341, 59028, 67535, 14390, 35960 and 29885, which map as follows:

1. 35341 mod 6 = 1: the root splits on B, creating `B0` to `B3`, each followed by a `<node>`;
2. 59028 mod 6 = 0: the node under `B0` splits on A, creating `A0` and `A1`;
3. 67535 mod 6 = 5: the node under `A0` is a leaf (`A0-END`);
4. 14390 mod 6 = 2: the node under `A1` splits on C, creating `C0` to `C4`;
5. 35960 mod 6 = 2: the node under `C0` splits on C again, which is later truncated;
6. 29885 mod 6 = 5: the first child of that repeated split is a leaf (`C0-END`).

This produces exactly the excerpt shown above. Each codon decides one node, so the number of codons used grows with the size of the hierarchy, and a derivation fails if it reaches the end of the genome before all nodes are decided (Section 3.6).

3.4.4 From Grammar to Series

The phenotype is parsed by `pheno_to_dict` (`src/fitness/mobai_utils.py`), which reads it line by line and uses the indentation to keep track of the path from the root. Each line adds the condition (feature, value) of its label to the path, a line ending in `END` closes a leaf, and the conditions accumulated on a root-to-leaf path become the filter of that leaf. If a feature repeats along a path, the branch is truncated as explained above and the parent is kept as a leaf. The output is a list of filters, one per leaf; for the excerpt above, the first two are

$$\phi_1 = \{\text{Event Type} = \text{ORIGINATED},\ \text{Call Type} = \text{SMS}\},$$
$$\phi_2 = \{\text{Event Type} = \text{ORIGINATED},\ \text{Call Type} = \text{VOICE},\ \text{Subscriber Type} = \text{Postpaid}\}.$$

`hierarchy_auc.evaluate` then calls `create_final_hierarchy`, which builds the series of Equation (3.2) for each leaf. For every filter, `filter_and_aggregate` selects the matching records and sums #Events and the anomaly flags per hour. `set_leaf` then aligns the series to the full hourly index of the split (training or test): hours with no matching records are added with #Events = 0, and an hour is labelled anomalous if at least one of its records was flagged. Alignment is needed because many segments have no records in many hours; without it, consecutive values of a series could be several hours apart and the 168-point windows of the MP would no longer correspond to one week.

A leaf whose filter matches no records produces an empty series. This happens often, since only 77 of the 240 combinations of the five features occur in the data. Empty leaves carry no traffic and no anomalies, and they are ignored by the fitness function.

3.5 Fitness Evaluation

Each candidate hierarchy is evaluated by `hierarchy_auc.evaluate`, which converts the phenotype into one hourly #Events series per leaf (Section 3.4.4), scores each series with the Matrix Profile, and combines the per-leaf ROC-AUC values into a single weighted fitness. The fitness answers the question: if an operator monitored these series with the MP, how well would the scores rank the anomalous hours above the normal ones?

3.5.1 Matrix Profile Scoring

For each leaf series, `leaf_scores` computes the Matrix Profile with `stumpy.stump(ts, m=168, ignore_trivial=True)`, i.e. with a subsequence length of $m = 168$ hours (one week). As explained in Chapter 2, the value of the profile at position j is the z-normalised Euclidean distance between the week that starts at hour j and its most similar week elsewhere in the series; trivial matches, which overlap the week itself by more than three quarters, are excluded (`ignore_trivial`). A high value marks a discord: a week whose shape has no close match. The MP needs no labels and no training, and it has a single parameter, the window length, which is fixed to one week to match the weekly cycle of the traffic.

The MP is defined per subsequence, whereas the labels are defined per hour, so the profile has to be converted into an hourly score. An anomaly at hour t changes every week that contains t, that is, the 168 subsequences starting at hours $t - 167$ to t; the raw profile is therefore raised over a plateau of 168 positions, and does not indicate which hour of the plateau is anomalous. The score used here is the first difference of the profile,

$$\text{score}(t) = P(t - m + 1) - P(t - m), \qquad t = m, \ldots, N - 1, \tag{3.5}$$

where $P(j)$ is the profile value of the subsequence starting at hour j and N is the length of the series. The subsequence starting at $t-m+1$ is the first one whose last hour is t, so the score measures how much the distance increases when hour t enters the window. For an isolated anomaly this increase is largest exactly at the anomalous hour, which turns the plateau into a peak at the right position. The scores are then min–max normalised to $[0, 1]$ within each leaf. Since no complete window ends before hour m, the first 168 hours of each series are not scored and their labels are ignored: the ground truth used for the ROC-AUC is the anomaly flag from hour 168 onwards ($y_{\text{true}} = \text{Anomaly}[m:]$).

3.5.2 Weighted AUC

For each leaf, the ROC AUC measures how well the score ranks the anomalous hours above the normal hours: it is the probability that a randomly chosen anomalous hour has a higher score than a randomly chosen normal hour. An AUC of 1 is a perfect ranking and 0.5 is no better than random. The AUC does not require a detection threshold, which is convenient during the search, when no threshold has been chosen yet.

`calculate_AUC_hierarchy` aggregates the leaf scores. A leaf ℓ is valid only if its labels after the first 168 hours contain at least two classes; let $\mathcal{L}_v$ be the set of valid leaves and E_ℓ the total #Events of leaf ℓ. The fitness is

$$F = \sum_{\ell \in \mathcal{L}_v} w_\ell \, \text{AUC}_\ell, \qquad w_\ell = \frac{E_\ell}{\sum_{k \in \mathcal{L}_v} E_k}, \tag{3.6}$$

where AUC_ℓ is the ROC-AUC of the Matrix Profile score of leaf ℓ against its anomaly flags.

The weights reflect operational importance: a leaf that carries more traffic counts more, and a tiny leaf with a lucky AUC cannot dominate the fitness. They also make the fitness comparable across hierarchies with very different numbers of leaves, since the weights always sum to one. The validity rule is necessary because the AUC is undefined for a leaf with only one class. Its consequence is that leaves without anomalies, including the empty ones, are excluded from the sum, so false alarms raised on them do not enter the fitness. This limitation is discussed in Chapter 4, where a pooled AUC that also counts these leaves is used as a robustness check.

3.5.3 Evaluation Procedure

Figure 3.8 summarises the evaluation of one individual. Individuals whose derivation fails are marked invalid by PonyGE2 before this function is called and receive no fitness (Section 3.6).

Require: phenotype of an individual, training records D_{train}, $m = 168$
1: $\{\phi_1, \ldots, \phi_L\} \leftarrow$ `pheno_to_dict`(phenotype) ▷ one filter per leaf
2: **for** $\ell = 1, \ldots, L$ **do**
3: $s_\ell, y_\ell \leftarrow$ hourly #Events and labels of the records matching ϕ_ℓ, aligned to the training hours
4: $\mathcal{L}_v \leftarrow \{\ell : y_\ell[m{:}] \text{ contains both classes}\}$
5: **for** $\ell \in \mathcal{L}_v$ **do**
6: $P \leftarrow$ `stumpy.stump`(s_ℓ, m); score $\leftarrow$ min–max(diff(P))
7: $\mathrm{AUC}_\ell \leftarrow$ `roc_auc_score`$(y_\ell[m{:}], \text{score})$
8: $w_\ell \leftarrow \sum_t s_\ell(t) \,/\, \sum_{k \in \mathcal{L}_v} \sum_t s_k(t)$
9: **return** $\sum_{\ell \in \mathcal{L}_v} w_\ell \, \mathrm{AUC}_\ell$

Figure 3.8: Fitness evaluation of a candidate hierarchy (`hierarchy_auc.evaluate`).

3.6 Genetic Programming Implementation

The genetic programming implementation is based on the PonyGE2 framework, which has the following key components: the grammar `grammars/mobai_nop.pybnf`, which generates candidate hierarchies over the five features A–E (Partner Description, letter F in `grammars/mobai.pybnf`, is not included); the fitness function `hierarchy_auc`, whose `evaluate` method calls `pheno_to_dict` and `create_final_hierarchy` to build the leaf series (Section 3.4.4) and `calculate_AUC_hierarchy` to return the weighted AUC of Equation (3.6); and the configuration parameters of Table 3.4. Figure 3.9 shows the control flow of PonyGE2: an initial population is generated and evaluated, and in every generation parents are selected, offspring are produced by crossover and mutation, evaluated, and replace the population.

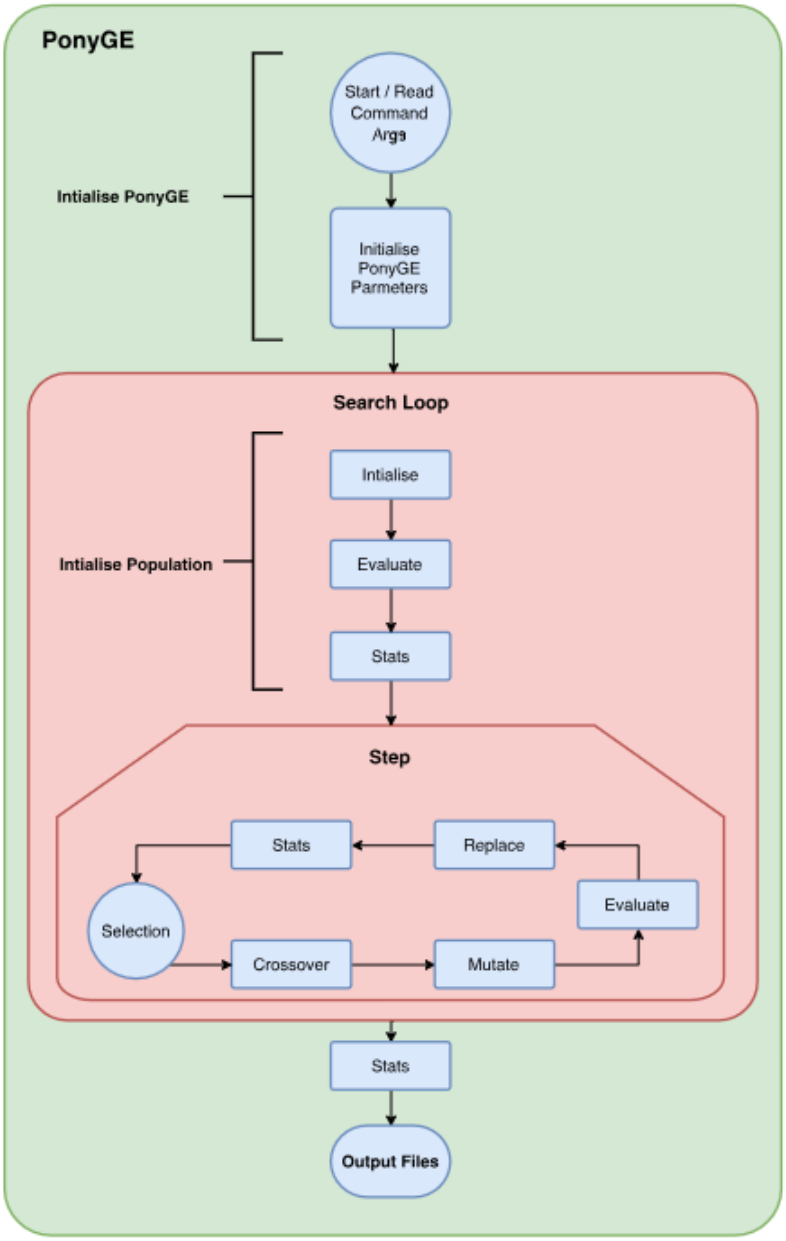


Figure 3.9: PonyGE2 control flow diagram (Fenton et al., 2017).

3.6.1 Genome Structure

The genome structure is defined by codons: each codon is an integer in the range $[0, 99{,}999]$, corresponding to a codon size of 100,000. Because the production is chosen with the modulo of the number of productions, which is 6 here, a large codon size makes every production equally likely for a random codon and lets mutation move freely between them. The maximum genome length (`MAX_GENOME_LENGTH`) is 500 codons and represents the maximum "DNA length" of an individual; it prevents the genomes from growing without bound through crossover, which can produce offspring longer than their parents. Genome wrapping is disabled (`MAX_WRAPS` $= 0$): when a derivation needs more codons than the genome contains, the individual is invalid instead of reusing the genome from the start. Invalid individuals receive no fitness and are not chosen as parents (`INVALID_SELECTION`

is false).

Since each codon decides one node, the length of the genome bounds the number of nodes of a hierarchy. The largest hierarchy splits every branch on all five features, starting with the feature with most values; it has $1+5+5\cdot4+5\cdot4\cdot3+5\cdot4\cdot3\cdot2+240 = 446$ nodes, excluding truncated branches, and the useful hierarchies found are much smaller. A cap of 500 codons is therefore above what such hierarchies require: it stops bloat without cutting off valid hierarchies. The derivation depth is further limited by `MAX_TREE_DEPTH` = 20. A useful hierarchy is at most five levels deep, since a feature repeated on a path is truncated, so this limit only constrains the truncated, non-functional branches.

3.6.2 Search Operators and Parameters

Table 3.4 lists the configuration of the experimental campaign and the role of each parameter; the operators themselves are described in Chapter 2.

Two parameters deserve a comment. Mutation is applied only to the used codons, that is, to the part of the genome that was read during the mapping, so every mutation changes a decision that is actually part of the hierarchy. Because codons are read depth first, a mutation near the start of the genome changes a split close to the root and alters the meaning of all the codons that follow; a mutation near the end changes only one small branch. The search therefore makes both large and small structural moves. Elitism guarantees that the best fitness never decreases from one generation to the next, so the best individual of the final generation is also the best individual found in the run; this is the hierarchy reported for each run in Chapter 4.

The population size and the number of generations are much smaller than the PonyGE2 defaults and than the values in `parameters/mobai.txt` (50 and 100). Each evaluation builds every leaf series from about 630,000 records and computes one MP per valid leaf, so the budget had to be reduced to complete ten independent runs (Section 3.7). PonyGE2 caches the fitness of every phenotype already evaluated, so duplicates produced by elitism or by neutral mutations of truncated branches are not scored twice.

3.7 Technological Tools and Computational Environment

This section describes the technological tools used in this dissertation, the parts of the implementation that were reused and developed, and the environment in which the experimental campaign was run. The whole pipeline, from data aggregation to the statistical tests, was implemented in Python. This choice follows from the two core components of the method: PonyGE2 (Fenton et al., 2017) and STUMPY (Law, 2019) are both Python libraries, so the search, the scorer and the evaluation share one language and one data representation. Table 3.5 lists the tools and the versions used in the experimental campaign.

Table 3.4: Key Configuration Parameters (experimental campaign)

Parameter	Value	Role in this work
CODON_SIZE	100000	Range of each codon, $[0, 99{,}999]$.
MAX_GENOME_LENGTH	500	Upper bound on the genome; prevents bloat.
MAX_WRAPS	0	No wrapping: an incomplete derivation is invalid.
INITIALISATION	PI_grow	Initial trees grown in random order, at least one branch reaching the maximum initial depth.
MAX_INIT_TREE_DEPTH	5	Initial hierarchies can already use all five features.
MAX_TREE_DEPTH	20	Limit on the depth of the derivation tree.
POPULATION_SIZE	10	Individuals per generation; small because of the cost of the fitness.
GENERATIONS	50	Number of generations, giving 510 evaluations per run.
SELECTION	tournament	Parents chosen by tournaments among random individuals.
TOURNAMENT_SIZE	2	Low selection pressure, which preserves diversity in a small population.
CROSSOVER	variable_onepoint	Tails of two parents swapped after independent cut points; offspring length may change.
CROSSOVER_PROBABILITY	0.75	Share of parent pairs that undergo crossover.
MUTATION	int_flip_per_codon	Each used codon replaced by a random integer with probability 1/(number of used codons), i.e. about one change per individual.
REPLACEMENT	generational	The offspring replace the population in every generation.
ELITE_SIZE	1	The best individual is copied unchanged to the next generation.
FITNESS_FUNCTION	hierarchy_auc	Weighted AUC of Equation (3.6) on the training period.

Table 3.5: Technological tools employed

Name	Version	Role in this work
Python	3.13.11	Programming language of the whole pipeline.
PonyGE2	fork	GE engine: genotype-to-phenotype mapping through the grammar, initialisation, selection, crossover, mutation, replacement and logging of each run.
STUMPY	1.14.1	Computation of the MP of each leaf series with `stumpy.stump`.
Numba	0.66.0	Just-in-time compiler used by STUMPY to run the MP kernels as native code.
pandas	2.3.3	Loading of the CSV datasets, filtering of the records of each leaf, hourly aggregation and alignment to the full hourly index.
NumPy	2.4.4	Array operations, including the first difference and min–max normalisation of the MP.
scikit-learn	1.8.0	ROC AUC (`roc_auc_score`) and precision, recall and F1 of the pooled evaluation.
SciPy	1.17.1	Wilcoxon signed-rank tests against the baselines.
Matplotlib	3.11.2	Figures of Chapter 4.
Kaggle Notebooks	–	Hosted Jupyter environment used for data preparation, exploratory analysis, enumeration of the feature combinations, anomaly injection and early prototypes of the hierarchy and MP scoring.

3.7.1 Reused and Developed Components

PonyGE2 is used as the search engine with its standard operators; the only changes to the framework replace the deprecated constant `np.NaN` with `np.nan` in `mapper.py` and `base_ff.py`, which is required by NumPy 2 and does not change its behaviour. The components specific to this problem were developed for this dissertation:

- the grammar `grammars/mobai_nop.pybnf`, which defines the space of admissible hierarchies (Section 3.4.2);
- the fitness function `src/fitness/hierarchy_auc.py`, which loads the training set once and evaluates each individual (Section 3.5);
- the module `src/fitness/mobai_utils.py`, which parses the phenotype (`pheno_to_dict`), builds and aligns the leaf series (`create_final_hierarchy`, `set_leaf`), scores them (`leaf_scores`) and computes the weighted AUC (`calculate_AUC_hierarchy`);
- the script `src/evaluate_experiment.py`, which re-evaluates the best individual of every run and the seven baselines on both periods, computes the original and pooled metrics and runs the Wilcoxon tests;
- the script `src/plot_results.py`, which produces the figures and tables of Chapter 4 from the saved runs.

The same functions are used during the search and in the final evaluation, so the reported test values are computed exactly as the fitness, only on the test period.

The data preparation that precedes the search was carried out in Kaggle notebooks over the private operator data. They load the raw extract, parse the timestamps, clean the #Events column, drop the unused columns and the duplicated records, and produce the exploratory analysis of Section 3.1. They also enumerate the 94,080 possible combinations of the six features and separate the 1,555 that occur in the data, with the number of hours in which each is observed; this list is what the injection procedure checks before accepting a segment. Finally, they implement the anomaly injection of Section 3.3 and the plots used to inspect the injected anomalies. The same notebooks also contain the early prototypes of the approach, in which hierarchies were sampled at random and scored first with a z-score heuristic and then with the MP; these prototypes motivated the replacement of random sampling by a grammar-guided search and are not part of the reported experiments.

3.7.2 Computational Environment

The experimental campaign was run on a desktop computer with an AMD Ryzen 7 8700F processor (8 cores, 16 threads) and 32 GB of RAM, under Windows 11. Ten independent runs were executed, one per random seed from 0 to 9, with four runs in parallel. Within each run, PonyGE2 evaluates the individuals of a generation on four processes (`MULTICORE`, `CORES` = 4) and caches the fitness of phenotypes already evaluated (`CACHE`), so duplicated individuals are not scored again. Each run took between 13 and 20 minutes. Most of this time is spent in the fitness function, since every evaluation filters the records and computes one MP per leaf; this cost is the reason for the reduced population size and number of generations (Section 4.1).

3.7.3 Reproducibility

Every run is identified by its random seed, and PonyGE2 stores in its result folder the full parameter file used (`parameters.txt`), the statistics of every generation (`stats.tsv`) and the best individual with its phenotype (`best.txt`). The campaign settings (population 10, 50 generations) override the defaults of `parameters/mobai.txt`, so the stored `parameters.txt` of each run is the reference for its configuration.

The final evaluation reads only these folders and the fixed training and test sets (`datasets/Mobai/train_set.csv` and `test_set.csv`), and writes the results to `evaluation.csv`, `evaluation_pooled.csv` and `results_tables.md`, so all values reported in Chapter 4 can be regenerated without repeating the search.

Chapter 4

Results and Discussion

This chapter reports the experimental evaluation of the aggregation hierarchies evolved with GE. Section 4.1 summarises the experimental configuration. Section 4.2 compares the evolved hierarchies with seven fixed aggregation baselines under two scoring schemes. Section 4.3 illustrates the behaviour of the detector on individual series, Section 4.4 analyses the search process and the evolved structures, and Section 4.5 discusses the findings. Section 4.6 closes the chapter with the threats to validity.

4.1 Experimental Setup

4.1.1 Data, Split and Labels

The experiments use the hourly #Events series of the switch data described in Chapter 3, after synthetic anomaly injection (Figure 3.3). The data is split temporally, without shuffling:

- **Training period:** 18 July 2022 00:00 to 14 September 2022 18:00 (1,411 hours);
- **Test period:** 14 September 2022 19:00 to 13 October 2022 21:00 (699 hours).

For every leaf of a hierarchy, the filtered records are aggregated to an hourly series and aligned to the full hourly range of the split, with missing hours filled with zero. Sparse feature combinations therefore become series that are zero for most hours.

Each injected anomaly modifies a single hour: the 168-hour window is only used to estimate the local mean μ and standard deviation σ, and the modified value is placed outside $\mu \pm 3\sigma$. The labels are therefore *point* anomalies. Because each anomaly affects one feature combination, the share of anomalous hours depends on the granularity of the view. In the test period it is 4.3% for the fully aggregated series, between 1.6% and 2.7% for the single-category views, 0.8% on average for the evolved hierarchies and 0.12% for full disaggregation.

4.1.2 Anomaly Scoring and Evaluation Metrics

Each leaf series is scored with the MP computed by STUMPY (Law, 2019) with a window of $m = 168$ hours (one week). The anomaly score of each hour is the first difference of the matrix profile, min–max normalised to $[0, 1]$. Since no complete window exists for the first m hours of a split, these hours are not scored and their labels are ignored.

Two metrics are reported. The first, used as fitness during the search, is the event-weighted mean of the leaf AUCs:

$$\mathrm{AUC}_H = \sum_{\ell \in \mathcal{V}} w_\ell \, \mathrm{AUC}_\ell, \qquad w_\ell = \frac{E_\ell}{\sum_{k \in \mathcal{V}} E_k}, \tag{4.1}$$

where AUC_ℓ is the ROC AUC of leaf ℓ, E_ℓ its total number of events, and $\mathcal{V}$ the set of leaves that contain both normal and anomalous hours after the first m hours. Leaves without anomalies are excluded from $\mathcal{V}$, so false alarms raised on them do not affect the metric. This is referred to as the *original* metric.

As a robustness check, a *pooled* AUC is also computed: the scores and labels of all leaves are concatenated into a single ROC curve, so anomaly-free leaves contribute negatives and their false alarms are counted. On the same pooled scores, the hours in the top 5% of scores are flagged as anomalous, and precision, recall and F1 are reported. The 5% threshold matches the injection rate and is deliberately generous given the label rates above.

4.1.3 Search Configuration

The search uses PonyGE2 (Fenton et al., 2017) with the grammar `mobai_nop.pybnf`, which covers five categorical features (Call, Event, Subscriber, Traffic and Service Type). Partner Description is excluded because of its high entropy (8.4 bits, Chapter 3). Table 4.1 lists the configuration used in the experimental campaign. Population size and number of generations were reduced to 10 and 50 because of the computational budget: each fitness evaluation builds and scores every leaf of a hierarchy. Each run took between 13 and 20 minutes with four runs in parallel.

Table 4.1: Configuration of the experimental campaign

Parameter	Value
Grammar	`mobai_nop.pybnf` (5 features)
Population size	10
Generations	50 (510 evaluations per run)
Independent runs	10 (random seeds 0–9)
Initialisation	PIG, maximum initial depth 5
Maximum tree depth	20
Maximum genome length	500 codons
Codon size	100,000
Selection	Tournament, size 2
Crossover	Variable one-point, probability 0.75
Mutation	Integer flip per codon
Replacement	Generational, with elitism
Fitness	AUC_H (Equation 4.1) on the training period

For each run, the best individual of the final generation is evaluated on both periods. When a single evolved hierarchy is analysed in detail, the run with the highest *training* fitness is used (seed 7); selecting by test performance would leak information from the test set.

4.1.4 Baselines and Statistical Testing

The evolved hierarchies are compared with seven fixed hierarchies:

- **Fully aggregated:** a single series with all traffic;
- **By Call Type** (2 leaves), **By Event Type** (4), **By Subscriber Type** (5), **By Traffic Type** (3) and **By Service Type** (2): one split on a single feature;
- **Fully disaggregated:** the 1,555 non-empty feature combinations, including Partner Description.

The single-feature baselines can also be generated by the grammar, so the search could, in principle, reach them.

The baselines are deterministic and are evaluated once. The ten evolved hierarchies are compared with each baseline using a two-sided one-sample Wilcoxon signed-rank test on the differences in test AUC. With ten runs, the smallest attainable p-value is 0.002, which occurs when all ten runs lie on the same side of the baseline.

4.2 Performance Analysis

4.2.1 Comparison with Fixed Baselines

Table 4.2 and Figures 4.1 and 4.2 report the original metric on the training and test periods. The evolved hierarchies reach a test AUC of 0.678 ± 0.066 (median 0.655, range 0.598–0.778). They are significantly better than four baselines: the fully aggregated series and the Call, Event and Service Type splits, by 0.10 to 0.14 AUC ($p = 0.002$). They are statistically indistinguishable from the Traffic Type split ($p = 0.92$), and worse than the Subscriber Type split (-0.056, $p = 0.027$) and full disaggregation (-0.198, $p = 0.002$).

Table 4.2: ROC-AUC of the best evolved hierarchy per run and of the fixed baselines, using the event-weighted metric over leaves containing anomalies. For the evolved hierarchies, values are mean ± standard deviation over 10 runs; Δ is the mean difference in test AUC (evolved minus baseline). The best value of each metric is in bold.

Hierarchy	Leaves	Train AUC	Test AUC	Δ	Wilcoxon p
GE (10 runs)	8–34	0.765 ± 0.019	0.678 ± 0.066	–	–
Fully aggregated	1	0.601	0.543	+0.135	0.002
By Call Type	2	0.683	0.562	+0.116	0.002
By Event Type	4	0.669	0.580	+0.098	0.002
By Service Type	2	0.642	0.574	+0.104	0.002
By Traffic Type	3	0.614	0.684	−0.006	0.92
By Subscriber Type	5	0.628	0.734	−0.056	0.027
Fully disaggregated	1,555	**0.791**	**0.877**	−0.198	0.002

Two baselines, Subscriber Type and Traffic Type, score higher on the test period than on the training period. The two periods therefore differ in how detectable the injected anomalies are, which is relevant to the generalisation analysis below.

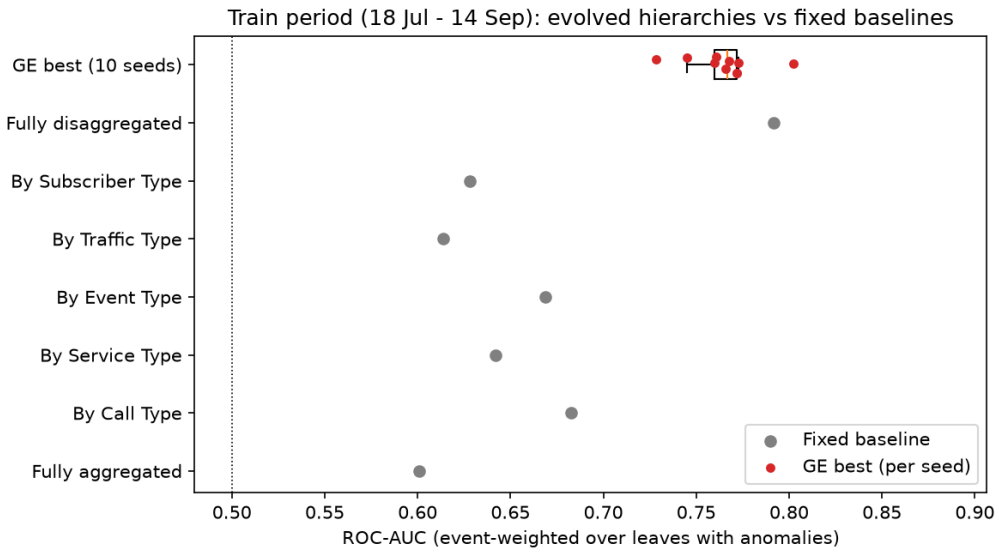


Figure 4.1: Training AUC (18 July–14 September 2022) of the fixed baselines (grey circles) and distribution of the training AUC of the best evolved hierarchy over 10 runs (box plot and red points).

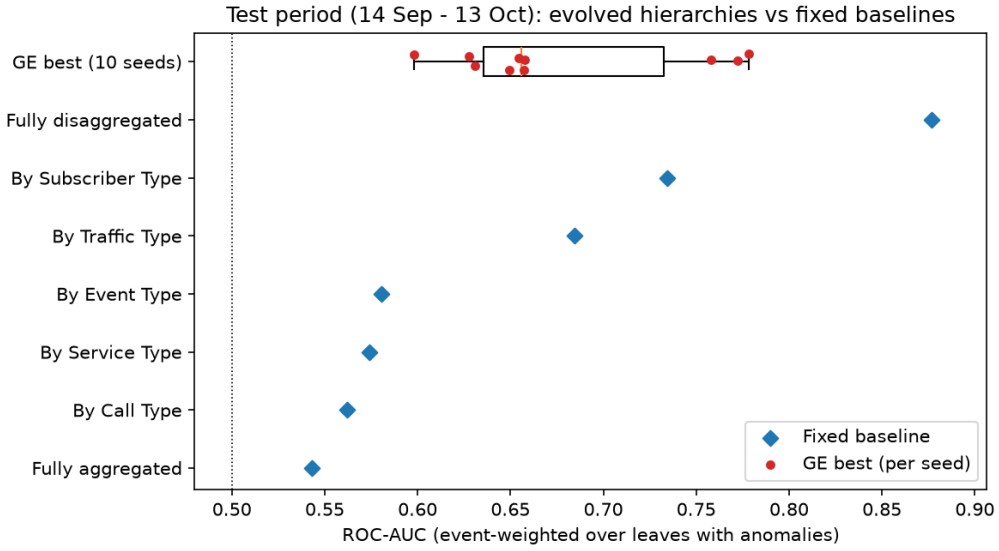


Figure 4.2: Test AUC (14 September–13 October 2022) of the fixed baselines (blue diamonds) and distribution of the test AUC of the best evolved hierarchy over 10 runs (box plot and red points). Rows and horizontal axis are the same as in Figure 4.1.

4.2.2 Robustness Check: Pooled Scoring

Table 4.3 and Figure 4.3 report the pooled metric, in which false alarms on anomaly-free leaves are counted. The ranking of the hierarchies changes substantially. Full disaggregation drops from 0.877 to 0.600, and its precision among the top 5% of scores is only 0.4%: roughly 250 false alarms are raised for each correctly flagged anomalous hour. The Subscriber Type split drops from 0.734 to 0.581, while the Event Type and Service Type splits rise to about 0.72.

Table 4.3: Pooled ROC-AUC over all leaf-hours and precision, recall and F1 when the top 5% of scores are flagged (test period). For the evolved hierarchies, values are means over 10 runs; p is the Wilcoxon signed-rank p-value of the difference Δ. The best value of each metric is in bold.

Hierarchy	Pooled AUC	Δ	p	Precision	Recall	F1
GE (10 runs)	0.670 ± 0.039	–	–	0.050	0.308	0.086
Fully aggregated	0.543	+0.127	0.002	0.107	0.130	0.118
By Call Type	0.510	+0.160	0.002	0.056	0.103	0.072
By Event Type	0.715	−0.046	0.010	0.083	0.220	0.121
By Service Type	**0.717**	−0.048	0.010	0.074	0.154	0.100
By Traffic Type	0.690	−0.021	0.16	**0.125**	**0.323**	**0.180**
By Subscriber Type	0.581	+0.088	0.002	0.060	0.186	0.091
Fully disaggregated	0.600	+0.069	0.002	0.004	0.165	0.008

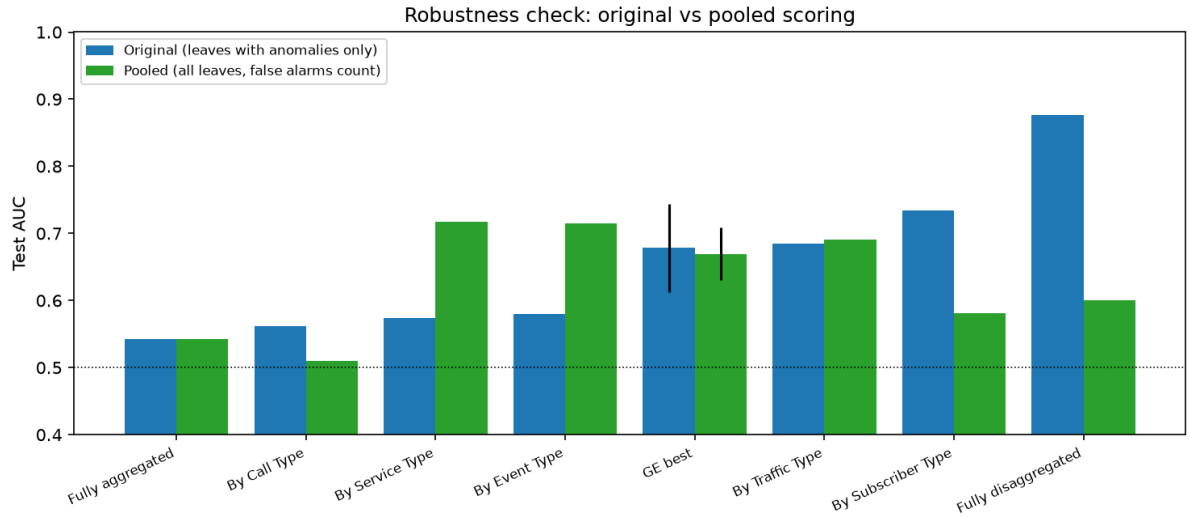


Figure 4.3: Test AUC under the original metric (leaves with anomalies only) and the pooled metric (all leaves). Error bars show the standard deviation over the 10 evolved hierarchies.

The mean pooled AUC of the evolved hierarchies (0.670) is close to their original test AUC (0.678). Under pooled scoring they remain significantly better than the fully aggregated series, the Call and Subscriber Type splits and full disaggregation, are indistinguishable from the Traffic Type split ($p = 0.16$), and are slightly worse than the Event

and Service Type splits (-0.05, $p = 0.010$). The ordering of the runs, however, changes: seeds 6 and 7, the best two runs under the original metric, have the lowest pooled AUC (0.606 and 0.623), while seeds 0, 1 and 9 perform best (0.70 to 0.72).

F1 is low for every hierarchy (at most 0.18). This follows from the label rates of 0.1% to 4.3% of hours and the 5% threshold, which bounds precision from above, and F1 is therefore not used as the main comparison.

4.2.3 Hierarchy Size and Detection Quality

Figure 4.4 relates the number of leaf series to the test AUC. The evolved hierarchies contain between 8 and 34 leaves (19.6 on average). Under the original metric, the only hierarchy that clearly outperforms them requires 1,555 series. Under pooled scoring, however, two single-feature splits with two to four leaves perform as well or slightly better.

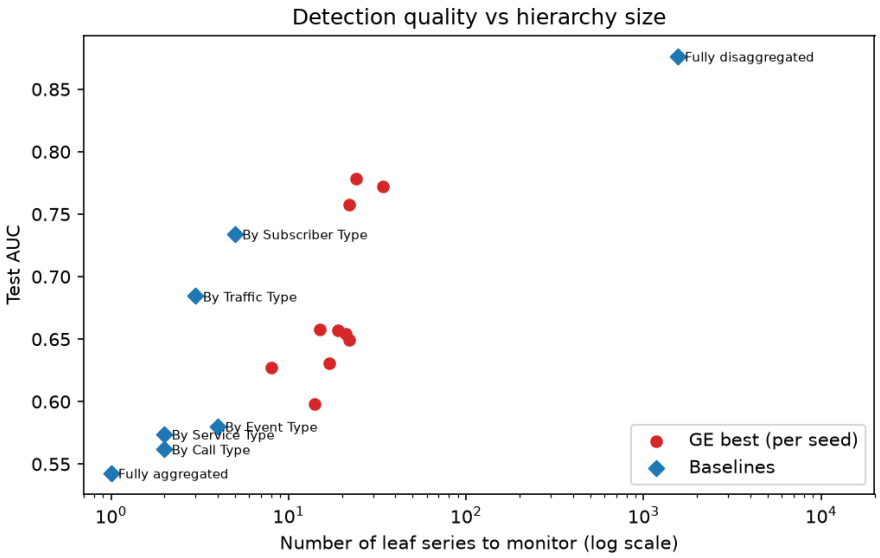


Figure 4.4: Test AUC (original metric) against the number of leaf series to monitor, on a logarithmic scale.

4.2.4 Generalisation

Figure 4.5 compares training and test AUC. Seven of the ten runs lose between 0.10 and 0.15 AUC on the test period, while seeds 6, 7 and 9 hold up (0.76 to 0.78). Training fitness is a weak guide to test performance: across runs, the Spearman correlation between training AUC and pooled test AUC is -0.27, and between original and pooled test AUC it is 0.20; neither is significant.

4.3 Time Series Visualisation

Figure 4.6 shows the test period for three leaves of the best evolved hierarchy (seed 7) and for the fully aggregated series. The three leaves were selected by rank, as the leaves with the best, median and worst leaf AUC, rather than chosen for visual appeal.

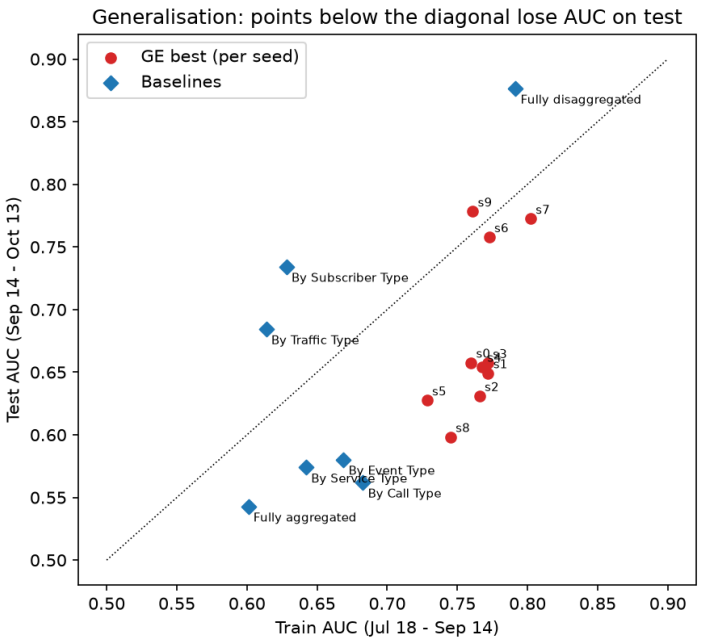


Figure 4.5: Training against test AUC (original metric) for each evolved hierarchy and baseline. Points below the diagonal lose AUC on the test period.

- **Best leaf** (TERMINATED / ON-NET / VOICE / Prepaid, AUC 1.00): a high-volume, regular series. The injected value is a clear discord and the score peaks at the labelled hour.
- **Median leaf** (TERMINATED / INT / Other, AUC 0.73): a noisier series in which some anomalies are detected and others are missed.
- **Worst leaf** (ORIGINATED / VOICE / Roamer In / OFF-NET, AUC 0.03): a sparse series with 0 to 4 events per hour. The injected value is indistinguishable from the normal spikes, so the MP cannot separate it.
- **Fully aggregated series** (AUC 0.54): many hours are labelled, but each anomaly originates in a small subgroup and is hidden in the total. This illustrates the dilution effect that motivates searching for an intermediate level of aggregation.

Two properties of the scorer are also visible. Anomalies in the first week are never scored, because the score starts at hour 168. Because the score is a first difference of the matrix profile, it produces a positive spike where a discord begins and a drop where it ends.

4.4 Genetic Programming Analysis

4.4.1 Fitness Evolution

Figure 4.7 shows the best and average training fitness per generation. The mean best fitness rises from 0.710 in the initial population to 0.765 after 50 generations, and the population average from 0.658 to about 0.73. Most of the improvement happens in the

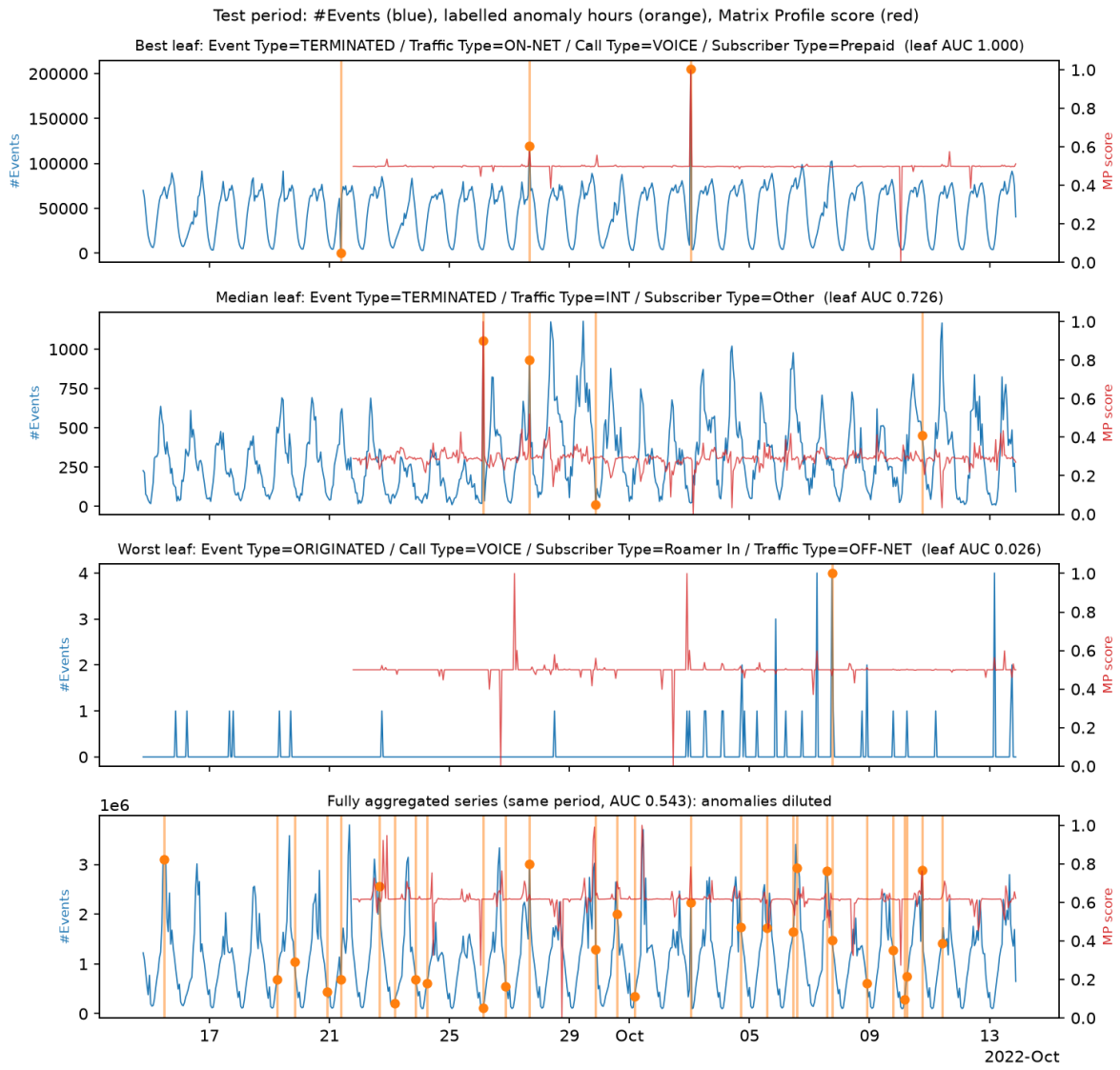


Figure 4.6: Test period for the best, median and worst leaf of the best evolved hierarchy (seed 7) and for the fully aggregated series. Blue: hourly #Events; orange: labelled anomalous hours; red: normalised MP score ($m = 168$).

first eight generations and is followed by long plateaus, as expected for a population of ten individuals with tournament size 2. The search had not converged: several runs still improve after generation 40, for example seed 7 at about generation 43. More generations would likely increase training fitness, but, given the generalisation gap reported above, not necessarily test performance.



Figure 4.7: Best training fitness per generation for each of the 10 runs (faint lines), their mean (solid) and the mean population average (dashed).

4.4.2 Best Evolved Hierarchy

Figure 4.8 shows the hierarchy evolved by seed 7, which has the highest training fitness (0.802; test 0.772). It contains 34 leaves, splits first on Event Type, and uses up to five conditions per leaf.

- The tree is asymmetric: it splits deeply where traffic is heterogeneous (ORIGINATED / VOICE and TERMINATED / INT) and stops early where it is not (ROAMING OUT has a single further split).
- The CALL FORWARD branch yields near-perfect leaves (AUC 0.95 to 1.00), but these carry little weight because the volume is concentrated elsewhere.
- The three worst leaves all belong to Roamer In traffic (AUC 0.03, 0.12 and 0.41); these are sparse, bursty series of the kind discussed in Section 4.3.
- A single leaf, TERMINATED / ON-NET / SMS, carries 43% of the weight with an AUC of 0.73, so the score of the hierarchy is largely the score of this series.
- Nine of the 34 leaves are not scored in the test period. Seven contain no anomaly at all and two contain an anomaly only in the unscored first week. Their false alarms do not affect the original metric.

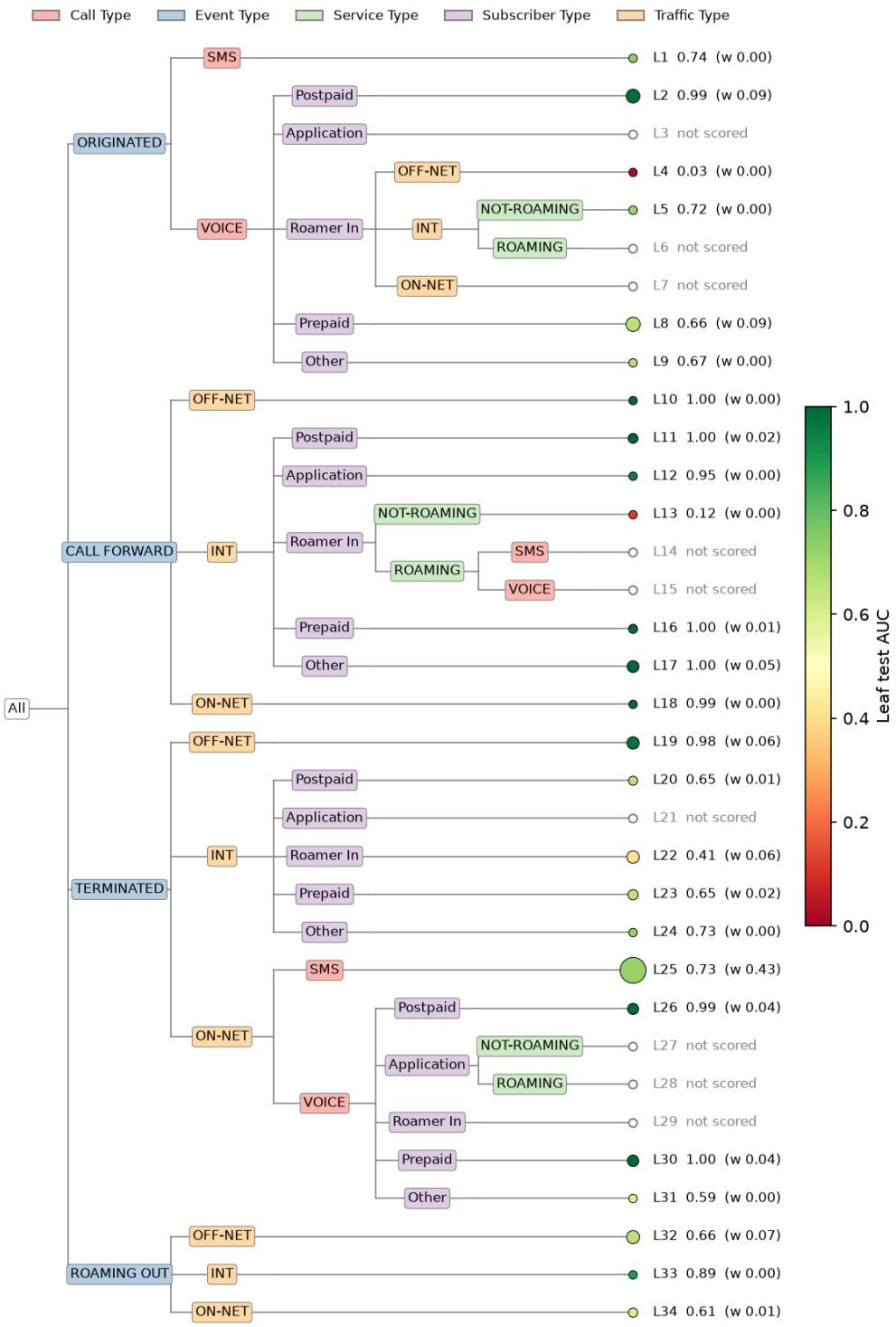


Figure 4.8: Hierarchy evolved by seed 7. Each path from the root to a leaf is a conjunction of filters, with boxes coloured by feature. Leaves are coloured by test AUC and sized by their weight w_ℓ in Equation 4.1; leaves without both classes in the test period are not scored.

4.4.3 Structure Across Runs

Table 4.4 summarises the best hierarchy of each run.

Table 4.4: Structure of the best evolved hierarchy per run. Depth is the maximum number of conditions in a leaf; unused features are those of the five grammar features that never appear in a filter. Train, test and pooled test values are AUCs.

Seed	Leaves	Depth	Root split	Unused features	Train	Test	Pooled
0	15	3	Call Type	Service	0.760	0.658	0.720
1	22	4	Call Type	–	0.772	0.649	0.704
2	17	4	Call Type	Traffic	0.766	0.631	0.659
3	19	5	Call Type	–	0.772	0.657	0.681
4	21	4	Call Type	–	0.768	0.654	0.678
5	8	3	Call Type	Event, Traffic	0.728	0.627	0.653
6	22	4	Event Type	–	0.773	0.758	0.606
7	34	5	Event Type	–	0.802	0.772	0.623
8	14	4	Service Type	Traffic	0.745	0.598	0.649
9	24	5	Call Type	–	0.761	0.778	0.724

- Call Type is the root split in seven of the ten runs. With two values (SMS and VOICE), it is the cheapest first split.
- Every run uses Call Type and Subscriber Type. Traffic Type is absent from the three runs with the lowest test AUC (seeds 2, 5 and 8), which, together with the strong Traffic Type baseline, suggests that this feature carries useful structure.
- Leaves use three to five conditions. Five is the maximum by construction: there are five features, and a branch is truncated when a feature repeats along a path. Derivation trees of up to 20 levels therefore collapse to at most five effective levels.
- There is no common best structure. The hierarchies differ widely between runs, yet several reach similar fitness, which indicates a flat fitness landscape with many near-equivalent structures. This favours interpretability, since a readable structure can be chosen among equivalent ones, but limits the stability of the result.

4.5 Discussion

4.5.1 Research Objectives

The objectives in Chapter 1 can be answered as follows.

- **Can aggregation choice be framed as a combinatorial search over a grammar?** Yes. Every individual that maps successfully is a valid hierarchy, and PonyGE2 found hierarchies with 8 to 34 leaves within 510 evaluations per run.
- **Can a label-free MP scorer drive the search?** Partly. With the detector fixed, the choice of aggregation alone moves the test AUC between 0.54 and 0.88 under the original metric, so the scorer clearly discriminates between views. However, the event-weighted fitness can be exploited, as discussed in Section 4.5.2.

- **Do evolved hierarchies outperform fixed aggregation?** They robustly outperform the coarse fixed views (the fully aggregated series and the Call Type split) under both metrics. They do not outperform the best single-feature splits, and whether they outperform full disaggregation depends on the metric.

4.5.2 Sensitivity to the Evaluation Metric

The same hierarchies rank differently under the two metrics, which is the main methodological finding of this work. Under the original metric, full disaggregation is the best hierarchy (0.877); under pooled scoring it falls below the evolved hierarchies (0.600). The reason is the exclusion of anomaly-free leaves from Equation 4.1: with 1,555 leaves, most leaves contain no anomaly, and the false alarms raised on them are not counted. The precision of 0.4% among the top 5% of scores shows the operational cost hidden by the original metric.

The same mechanism affects the search. Seeds 6 and 7, which score highest under the original metric, score lowest under pooled scoring, which suggests that the fitness rewards hierarchies that concentrate anomalies in a few leaves and leave many normal-only leaves that drop out of the score. A fitness that counts false alarms in every leaf, such as the pooled AUC, avoids this and is cheap to compute.

4.5.3 Generalisation Gap

The training and test periods behave differently: the Subscriber Type split scores 0.628 on the training period and 0.734 on the test period. Because the search optimises the training period only, without a validation split, it selects structures that fit the anomalies injected in that period. A validation window or rolling-origin evaluation of the fitness, larger populations, or averaging the fitness over several injection seeds could reduce this gap.

4.5.4 Parsimony and Interpretability

The evolved hierarchies require about 20 monitored series, compared with 1,555 for full disaggregation, and under pooled scoring they are also more accurate. Each leaf is a readable conjunction of business filters, such as TERMINATED / ON-NET / SMS, so an alert can be traced directly to a traffic segment. The contribution of the search is therefore competitive detection with a small, interpretable set of series, found without knowing in advance which feature to split on, rather than the best detection overall.

4.5.5 Behaviour of the Matrix Profile Scorer

The MP scorer works well on large, regular series with strong weekly seasonality, reaching leaf AUCs of 1.00. It fails on sparse integer series, such as the Roamer In leaves. A likely cause is that z-normalised subsequences consisting mostly of zeros make every isolated spike look like a discord, so injected and normal spikes cannot be distinguished. This supports excluding very sparse feature combinations, for example with a minimum-support rule in the grammar.

4.5.6 Relation to Previous Work

Telecom anomaly detection studies typically evaluate detectors on a fixed view of the data (Gonzalez et al., 2024; Mefleh et al., 2025; Burgueno et al., 2020). The results above show that, with the detector held fixed, the view alone changes the test AUC by up to 0.33, which supports treating aggregation as a search problem. Hierarchical search (Wolff et al., 2022) and subspace mining (He et al., 2021) also search over structure, but neither uses a grammar or MP-based scoring. The MP literature presents the matrix profile as a strong, parameter-light discord detector (Yeh et al., 2017); the present results agree for dense series and add a failure case for sparse ones. The AUC values reported here are computed on synthetic point anomalies and are not directly comparable with F1 or range-based F1 scores obtained on expert labels in those studies.

4.6 Limitations

- Only synthetic point anomalies are evaluated; real incidents may be longer, collective or contextual.
- A single dataset and a single temporal split are used, without cross-validation over time.
- The search budget is small (population 10, 50 generations, 10 runs) and the search had not converged.
- Partner Description is excluded from the grammar but included in the fully disaggregated baseline, so that comparison is not like-for-like.
- The original fitness ignores false alarms in anomaly-free leaves, and event-volume weighting allows a single leaf to dominate (43% of the weight for seed 7).
- The 5% threshold for the threshold metrics is arbitrary relative to label rates of 0.1% to 4.3%.
- Labels in the first 168 hours of each split are never scored.
- Only one score definition, the first difference of the matrix profile, was tested; the raw matrix profile and its absolute difference were not compared.

Chapter 5

Conclusions and Future Works

5.1 Conclusions

This dissertation addressed a practical question in the monitoring of mobile networks: at which level of aggregation should telecommunications traffic be analysed to detect anomalies? A fully aggregated series hides anomalies that affect a small segment of the traffic, while one series per combination of categories produces too many sparse series to monitor and interpret. Instead of fixing the aggregation in advance, this work treated it as a combinatorial search. Candidate hierarchies were generated by a BNF grammar over the categorical features of the traffic, evolved with GE in PonyGE2, and ranked by how well a label-free Matrix Profile detector found synthetic anomalies in the series they produced.

The framework was applied to hourly #Events from a mobile network switch, covering July to October 2022, with about one million records and six categorical features. Because the operational data has no reliable anomaly labels, 105 point anomalies were injected at three standard deviations from the local weekly mean, each in a randomly chosen segment of the traffic. The data analysis showed that the traffic is highly concentrated: a few combinations carry most of the events, while most of the 1,555 non-empty combinations are small and sparse. Partner Description, with 392 values and the highest entropy, was therefore excluded from the grammar, which kept the search space manageable and avoided splits that produce hundreds of nearly empty series.

The main conclusions are the following.

- **Aggregation matters as much as the detector.** With the detector held fixed, the choice of aggregation alone changed the test AUC from 0.54 for the fully aggregated series to 0.88 for full disaggregation. The injection experiments also showed why: an anomaly that is obvious in its own segment is diluted, and often invisible, in any series that also contains other traffic. This supports the premise of the dissertation that the view of the data should be chosen, and not assumed.

- **A grammar is a practical way to define the search space.** The grammar generates only valid hierarchies, each leaf is a readable conjunction of business filters, and the leaves always partition the traffic, so no repair or penalty is needed. The space it defines contains about 2.2×10^{61} distinct hierarchies, which rules out exhaustive search, yet PonyGE2 found hierarchies with 8 to 34 leaves in 510 evaluations per run.

- **The evolved hierarchies are competitive, but not the best overall.** They reached a test AUC of 0.678±0.066 and were significantly better than the fully aggregated series and the Call, Event and Service Type splits. They did not outperform the best single-feature splits, and under the original metric full disaggregation was better. Their advantage is a compact, interpretable set of about 20 series that is found without knowing in advance which features to split on.

- **The evaluation metric changes the conclusions.** The event-weighted AUC used as fitness ignores leaves without anomalies, so false alarms on those leaves are not counted. When they are counted, with a pooled AUC, full disaggregation falls from 0.877 to 0.600, with a precision of 0.4%, while the evolved hierarchies keep a similar value (0.670). The two runs with the best original fitness had the lowest pooled AUC, which shows that the search can exploit this blind spot of the fitness function. This is the main methodological finding of the work.

- **The Matrix Profile works well on dense series and fails on sparse ones.** On high-volume series with a regular weekly shape, the first difference of the profile located the injected anomalies with leaf AUCs up to 1.00. On sparse series with a few events per hour, such as inbound roaming segments, every isolated spike looks like a discord and the detector performed worse than random. Which leaves a hierarchy creates therefore determines not only how visible the anomalies are but also whether the detector can work at all.

- **Training fitness was a weak guide to test performance.** Most runs lost 0.10 to 0.15 AUC on the test period, and the fitness landscape was flat, with very different hierarchies reaching similar fitness. The small search budget, ten individuals for 50 generations, also meant that the search had not converged.

The contributions of this dissertation are, first, the formulation of aggregation choice for anomaly detection in grouped telecommunications series as a grammar-based search, with a fitness built from a label-free detector and synthetic anomalies; second, an implementation in Python on top of PonyGE2 and STUMPY that evaluates candidate hierarchies on real operator data and can be reproduced from the saved runs; and third, an empirical analysis that compares evolved and fixed hierarchies under two metrics and shows that conclusions about the best aggregation depend strongly on how false alarms are counted. For a network operations team, the practical message is that monitoring at an intermediate level of aggregation, chosen with the data, gives a small number of interpretable series in which anomalies are more visible than in the total, as long as sparse segments are avoided.

5.2 Future Work

The results and limitations of this work suggest several directions.

- **A fitness that counts false alarms.** The most direct improvement is to use the pooled AUC, or another metric that includes anomaly-free leaves, as the fitness. It is cheap to compute and would remove the incentive, observed in the best runs, to create many normal-only leaves. Precision-oriented metrics, such as the precision–recall AUC or F1 at an operational alarm budget, are also worth testing.

- **Control of sparse leaves.** Since the Matrix Profile fails on sparse series, a minimum-support rule could stop a branch when its traffic falls below a threshold, either in the grammar or in the fitness function. A multi-objective formulation could additionally trade detection quality against the number of leaves, making the cost of monitoring an explicit objective.
- **A larger search budget and a study of stability.** With more individuals and generations, it would be possible to establish whether the search converges, how sensitive it is to the random seed, and whether the flat fitness landscape contains a stable core of useful splits.
- **Real incidents and other anomaly types.** Only synthetic point anomalies were evaluated. Validating the evolved hierarchies against real incident records from the operator, and injecting collective or contextual anomalies that last several hours, would show whether the conclusions hold for the anomalies that matter in operation.
- **More robust validation over time.** A single temporal split was used. Rolling-origin evaluation over several periods, and data from months beyond July to October 2022, would give a better estimate of how well an evolved hierarchy generalises and how often it needs to be re-evolved.
- **Reintroducing Partner Description.** The partner feature was excluded because of its cardinality. Grouping partners, for example keeping the most frequent ones and merging the long tail into a single category, would let the search use this information without creating hundreds of sparse branches.
- **Other scores and detectors.** Only the first difference of the Matrix Profile was tested. The raw profile and alternative detectors, such as forecast residuals or autoencoders, could be plugged into the same fitness function, keeping GE for the structure search and testing whether the best aggregation depends on the detector.
- **Additional targets.** Only #Events was used. Event Duration could be added as a second target, either as a separate search or as a second objective, since some faults may affect call length without changing the number of calls.
- **Towards operation.** Finally, the approach could be moved towards an online setting, in which the hierarchy is evolved periodically on recent data and the leaf series are scored as new hours arrive, with the evolved tree and the discord plots presented to operators as an explanation of each alert.

Bibliography

Athanasopoulos, George et al. (2017). "Forecasting with temporal hierarchies". In: *European Journal of Operational Research* 262.1, pp. 60–74.

Banzhaf, W. et al. (1998). *Genetic Programming: An Introduction*. Morgan Kaufmann.

Box, George EP et al. (2015). *Time series analysis: Forecasting and control*. 5th. John Wiley & Sons.

Bradley, Andrew P (1997). "The use of the area under the ROC curve in the evaluation of machine learning algorithms". In: *Pattern recognition* 30.7, pp. 1145–1159.

Burgueno, Jesus et al. (2020). "Online Anomaly Detection System for Mobile Networks". In: *Sensors* 20.24, p. 7232. DOI: 10.3390/s20247232.

Chandola, Varun, Arindam Banerjee, and Vipin Kumar (2009). "Anomaly detection: A survey". In: *ACM computing surveys (CSUR)* 41.3, pp. 1–58.

Fawcett, Tom (2006). "An introduction to ROC analysis". In: *Pattern recognition letters* 27.8, pp. 861–874.

Fenton, Michael et al. (2017). "Ponyge2: Grammatical evolution in python". In: *Proceedings of the genetic and evolutionary computation conference companion*, pp. 1194–1201.

Garey, M. R. and D. S. Johnson (1979). *Computers and intractability: A guide to the theory of NP-completeness*. W.H. Freeman and Company.

Glover, Fred and Harvey J Greenberg (1989). "New approaches for heuristic search: A bilateral linkage with artificial intelligence". In: *European Journal of Operational Research* 39.2, pp. 119–130.

Gonzalez, Guillermo Garcia et al. (2024). "One Model to Find Them All: Deep Learning for Multivariate Time-Series Anomaly Detection in Mobile Network Data". In: *IEEE Transactions on Network and Service Management* 21.2, pp. 1601–1616. DOI: 10.1109/TNSM.2023.3340146.

Hanley, James A and Barbara J McNeil (1982). "The meaning and use of the area under a receiver operating characteristic (ROC) curve." In: *Radiology* 143.1, pp. 29–36.

Hawkins, Douglas M (1980). *Identification of outliers*. Vol. 11. Springer.

He, Jieming et al. (2021). "Mining Anomalies in Subspaces of High-Dimensional Time Series for Financial Transactional Data". In: *Machine Learning and Knowledge Discovery in Databases. Applied Data Science Track*, pp. 19–36. DOI: 10.1007/978-3-030-86514-6_2.

Hong, Chi-Yao et al. (2012). "Tiresias: Online Anomaly Detection for Hierarchical Operational Network Data". In: *2012 IEEE 32nd International Conference on Distributed Computing Systems*, pp. 173–182. DOI: 10.1109/ICDCS.2012.30.

Hua, X. et al. (2022). "GenAD: General Unsupervised Anomaly Detection Using Multivariate Time Series for Large-Scale Wireless Base Stations". In: *Electronics Letters* 59.1. DOI: 10.1049/ell2.12683.

Huang, Ming et al. (2026). "HYDRA: A Multi-Level Hierarchy-Driven Approach for Robust Anomaly Detection in Time Series". In: *Proceedings of the ACM on Management of Data* 4.3, pp. 1–30. DOI: 10.1145/3802074.

Hyndman, R. J. and G. Athanasopoulos (2018). *Forecasting: Principles and Practice*. 2nd ed. Retrieved from https://otexts.com/fpp2/. OTexts.

— (2021). "Chapter 11 Forecasting hierarchical and grouped time series". In: *Forecasting: Principles and Practice*.

Keogh, Eamonn, Jessica Lin, and Ada Fu (2005). "HOT SAX: Efficiently Finding the Most Unusual Time Series Subsequence". In: *Fifth IEEE International Conference on Data Mining*, pp. 226–233. DOI: 10.1109/ICDM.2005.79.

Koza, J. R. (1992). *Genetic Programming: On the Programming of Computers by Means of Natural Selection*. MIT Press.

Law, Sean (2019). "STUMPY: A Powerful and Scalable Python Library for Time Series Data Mining". In: *Journal of Open Source Software* 4.39, p. 1504. DOI: 10.21105/joss.01504.

Lemaire, Pierre et al. (2024). "Early Detection of Critical Urban Events Using Mobile Phone Network Data". In: *PLOS ONE* 19.8, e0309093. DOI: 10.1371/journal.pone.0309093.

Linares-Barrera, Maria Luisa et al. (2024). "Evolutionary Feature Selection for Time-Series Forecasting". In: *Proceedings of the 39th ACM/SIGAPP Symposium on Applied Computing*, pp. 395–399. DOI: 10.1145/3605098.3636191.

Lobo, Jorge M, Alberto Jiménez-Valverde, and Raimundo Real (2008). "AUC: a misleading measure of the performance of predictive distribution models". In: *Global ecology and Biogeography* 17.2, pp. 145–151.

Mefleh, A. et al. (2025). "Beyond the Norm: Unsupervised Anomaly Detection in Telecommunications with Mahalanobis Distance". In: *Computers* 14.12, p. 561. DOI: 10.3390/computers14120561.

O'Neill, Michael and Conor Ryan (2001). "Grammatical Evolution". In: *IEEE Transactions on Evolutionary Computation* 5.4, pp. 349–358. DOI: 10.1109/4235.942529.

O'Neill, M. and C. Ryan (2003). *Grammatical Evolution: Evolutionary Automatic Programming in an Arbitrary Language*. Springer.

Oguz, H. T. and A. Kalaycioglu (2022). "Anomaly Detection in Multi-Tiered Cellular Networks Using LSTM and 1D CNN". In: *EURASIP Journal on Wireless Communications and Networking* 2022.1. DOI: 10.1186/s13638-022-02183-7.

Papadimitriou, C. H. and K. Steiglitz (1982). *Combinatorial optimization: Algorithms and complexity*. Prentice Hall.

Papadimitriou, Christos H and Kenneth Steiglitz (1998). *Combinatorial optimization: algorithms and complexity*. Courier Corporation.

Pietroń, Marcin et al. (2025). "AD-NEv: A Scalable Multilevel Neuroevolution Framework for Multivariate Anomaly Detection". In: *IEEE Transactions on Neural Networks and Learning Systems* 36.5, pp. 8939–8953. DOI: 10.1109/TNNLS.2024.3436086.

Ruff, Lukas et al. (2021). "A unifying review of deep and shallow anomaly detection". In: *Proceedings of the IEEE* 109.5, pp. 756–795.

Sultan, Kashif, Hazrat Ali, and Zhang Zhang (2018). "Call Detail Records Driven Anomaly Detection and Traffic Prediction in Mobile Cellular Networks". In: *IEEE Access* 6, pp. 41728–41737. DOI: 10.1109/ACCESS.2018.2859756.

Van Onsem, M. et al. (2023). "Variable Length Motif Discovery in Time Series Data". In: *IEEE Access* 11, pp. 73754–73766. DOI: 10.1109/ACCESS.2023.3295995.

Vazirani, Vijay V (2001). *Approximation algorithms*. Vol. 1. Springer.

Wickramasuriya, Shanika L, George Athanasopoulos, Rob J Hyndman, et al. (2015). "Forecasting hierarchical and grouped time series through trace minimization". In: *Department of Econometrics and Business Statistics, Monash University* 105, pp. 6–1.

Wickramasuriya, Shanika L, George Athanasopoulos, and Rob J Hyndman (2019). "Optimal forecast reconciliation for hierarchical and grouped time series through trace minimization". In: *Journal of the American Statistical Association* 114.526, pp. 804–819.

Wolff, Bar et al. (2022). "Composite Anomaly Detection via Hierarchical Dynamic Search". In: *2022 IEEE International Symposium on Information Theory*, pp. 2421–2426. DOI: 10.1109/ISIT50566.2022.9834631.

Xue, Bing et al. (2016). "A Survey on Evolutionary Computation Approaches to Feature Selection". In: *IEEE Transactions on Evolutionary Computation* 20.4, pp. 606–626. DOI: 10.1109/TEVC.2015.2504420.

Yeh, Chin-Chia Michael et al. (2017). "Time Series Joins, Motifs, Discords and Shapelets: A Unifying View That Exploits the Matrix Profile". In: *Data Mining and Knowledge Discovery* 32.1, pp. 83–123. DOI: 10.1007/s10618-017-0519-9.

Zhu, Yan et al. (2016). "Matrix Profile II: Exploiting a Novel Algorithm and GPUs to Break the One Hundred Million Barrier for Time Series Motifs and Joins". In: *2016 IEEE 16th International Conference on Data Mining*, pp. 739–748. DOI: 10.1109/ICDM.2016.0085.

Zlochin, Mark et al. (2004). "Model-based search for combinatorial optimization: A critical survey". In: *Annals of Operations Research* 131, pp. 373–395.